

CSI 2110
Data Structures and Algorithms

Lecture 5: Priority Queue and Heap

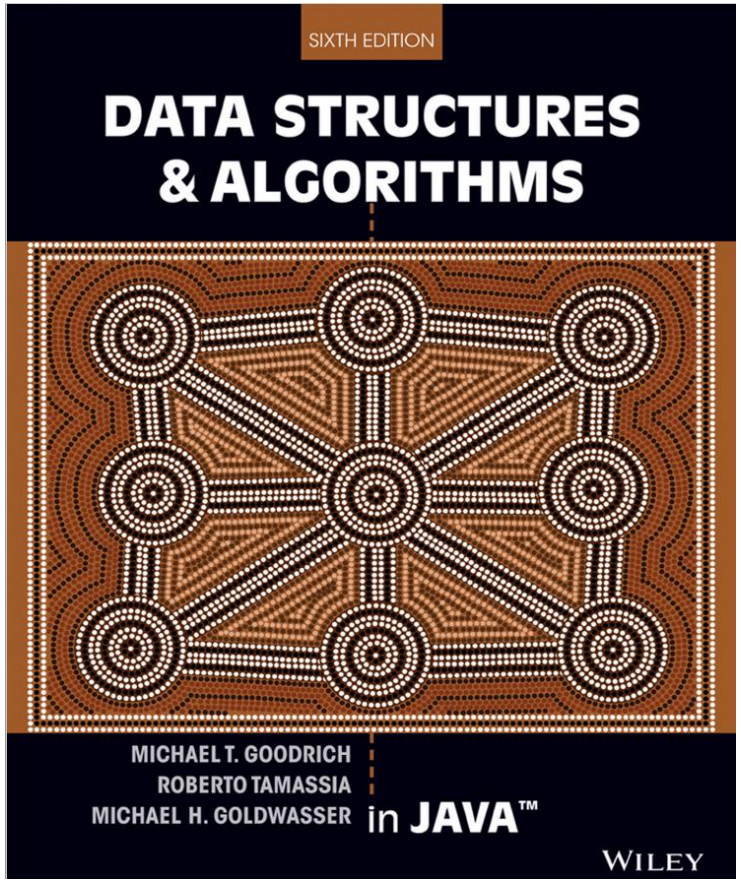
Bo Sun

Fall 2026

A note on the slides

Adapted from the slides that accompany this book.

Part of the slides is based on previous slides from Prof. Lucia Moura for CSI 2110.

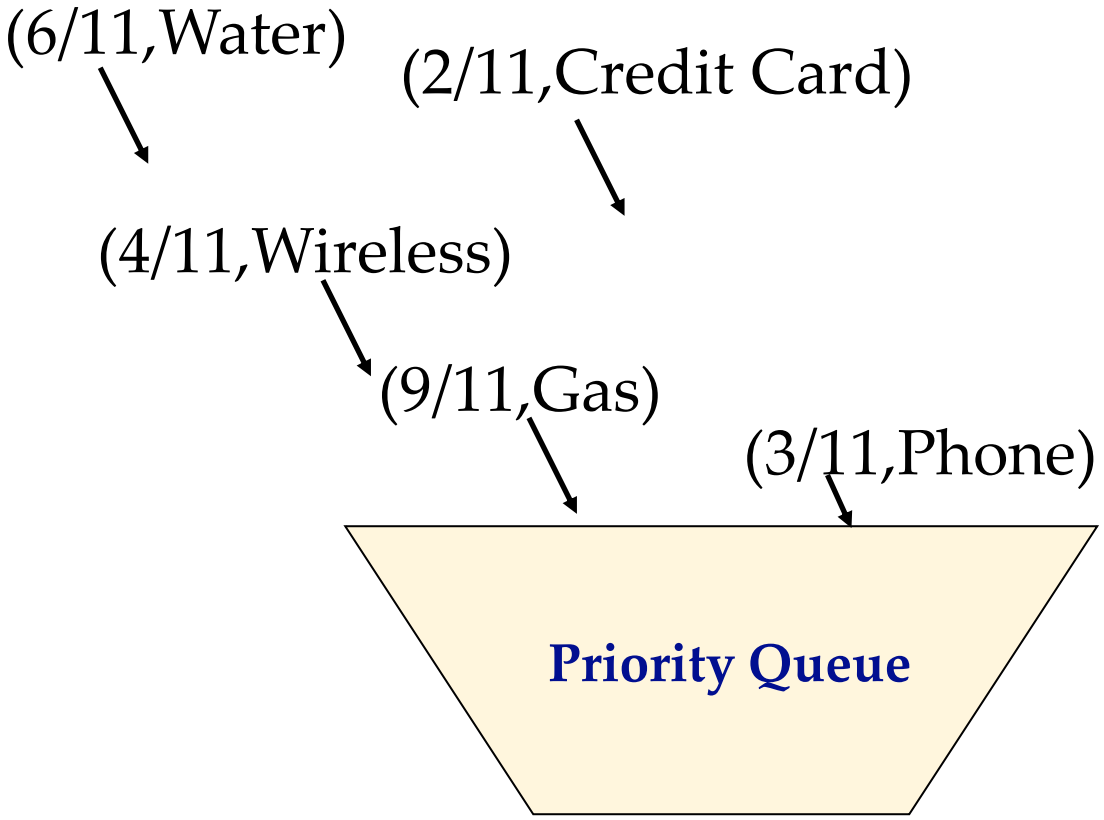


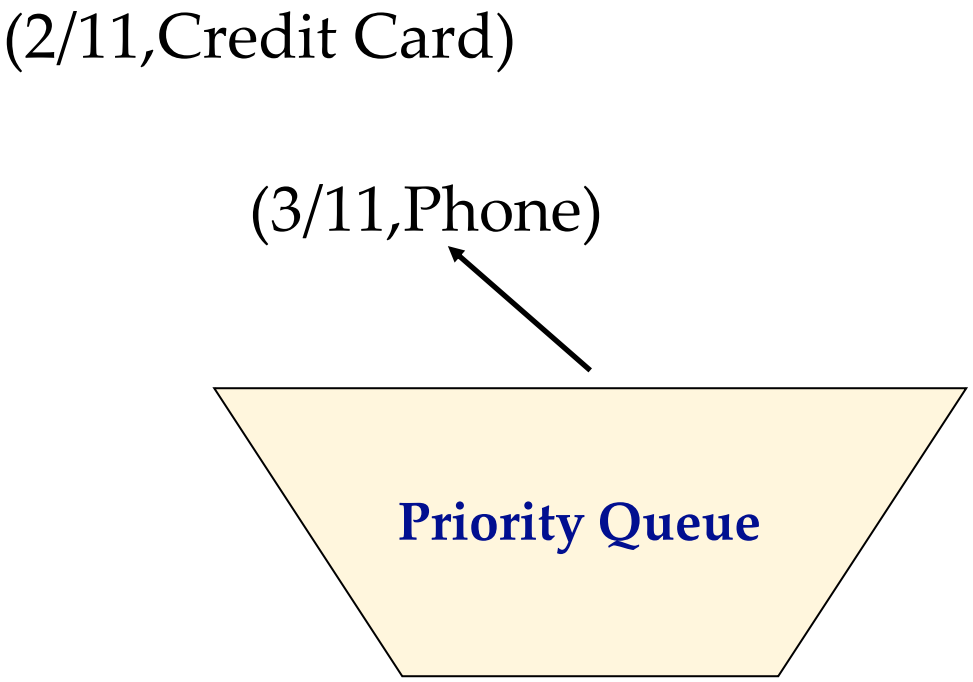
Data Structures and Algorithms in Java
6th edition
Michael Goodrich, Roberto Tamassia, Michael Goldwasser
Wiley, 2014

Priority Queue and Heap: roadmap

- Priority queue ADT
- Implementing priority queues using lists
  - Unsorted list
  - Sorted list
- Heap
  - Insert and delete methods
  - Heap construction
- Heap sort

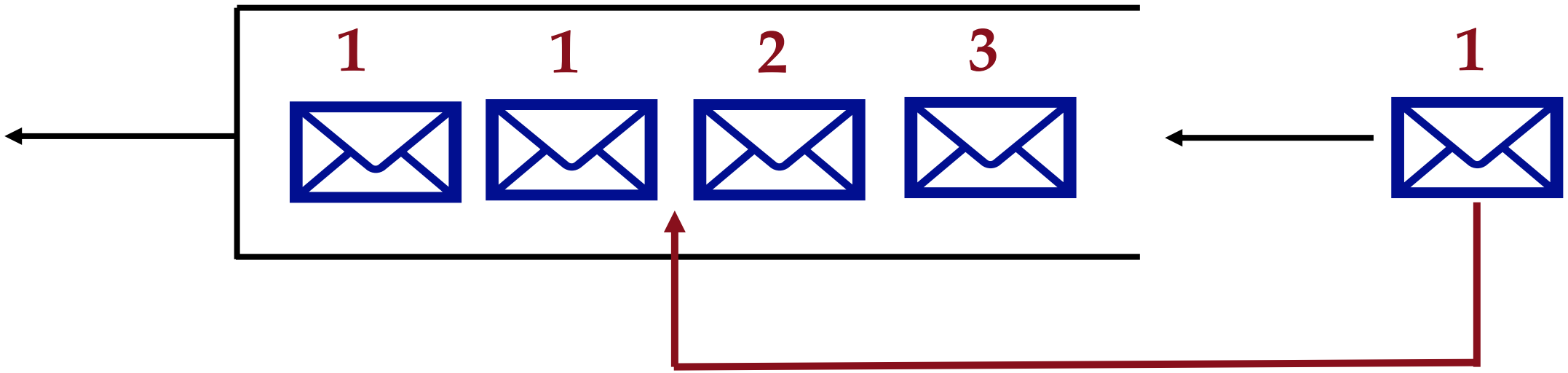
Lead-in example: deadline to pay a bill





Priority Queue (PQ)

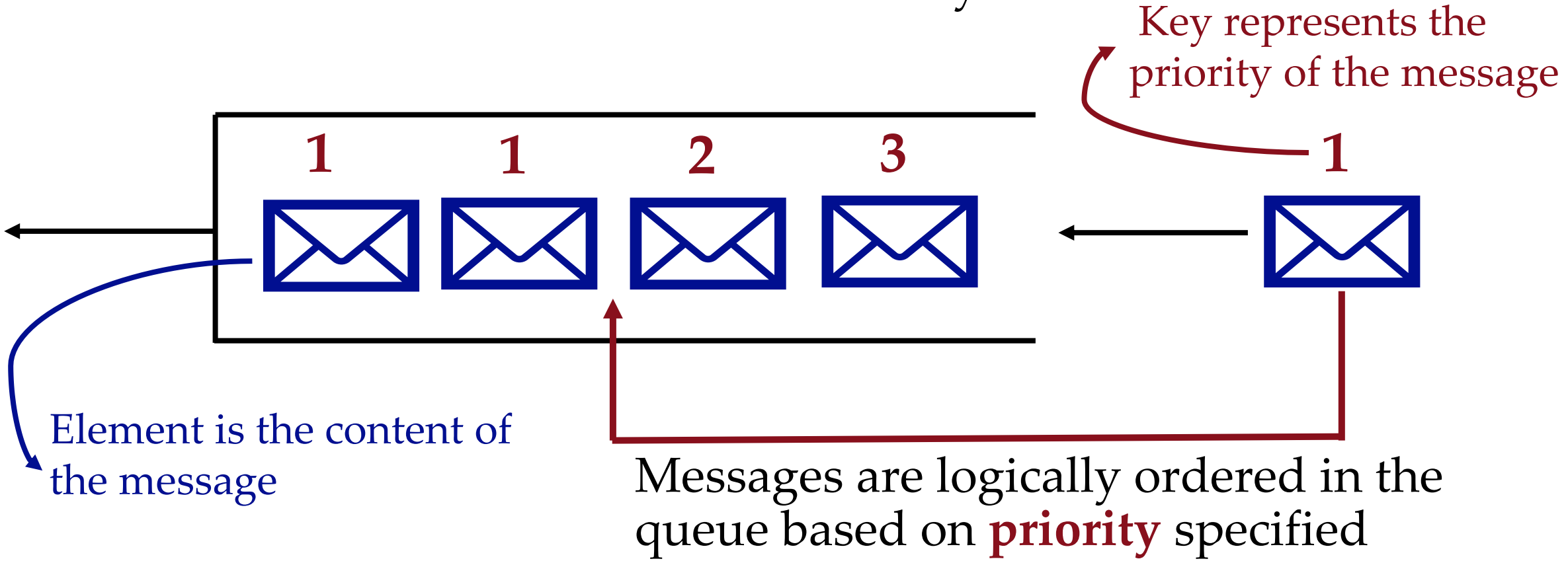
- A priority queue is a collection of elements, each associated with a priority
  - elements can be inserted in any order
  - the element with the highest priority is removed first



Messages are logically ordered in the queue based on **priority** specified

Priority Queue (PQ): (Key, Element)

- Each entry in a PQ is a (key, element) or (key, value) pair
  - keys can be any comparable objects
  - Two distinct entries can have the same key



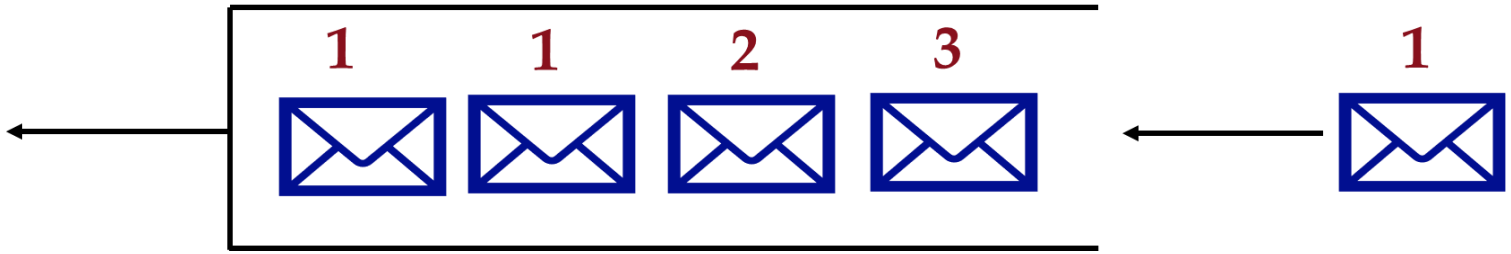
Priority Queue ADT

Main methods

- insert(k, v): insert an entry with key k and value v
- removeMin(): remove and return the entry (k, v) with the smallest key (or null if the priority queue is empty)

Support methods

- min(): return, but does not remove, an entry with the smallest key (or null if the priority queue is empty)
- size(), isEmpty()



Priority Queue and Heap: roadmap

- Priority queue ADT
- Implementing priority queues using lists
  - Unsorted list
  - Sorted list
- Heap
- Implementing priority queues using heaps
- Heap sort

Implementing priority queue: Entry ADT

```
public interface PriorityQueue<K,V> {
   insert(K key, V value);
   Entry<K,V> removeMin();
   /*other methods, e.g., min() */
}
```

- Priority queues store entries (key-value pairs) to allow for efficient insertion and deletion based on key
  - Use Entry interface

```
public class PQEntry<K,V> implements Entry<K,V> {
   private K key;
   private V value;
   public PQEntry(K k, V v) {
      key = k;
      value = v;
   }
   public K getKey() {return key;}
   public V getValue() {return value;}
}
```

Implementing priority queue: Comparator ADT

```
public interface Comparator<K> {

   int compare(K x, K y);
}
```

```
public class ReverseIntegerComparator
implements Comparator<K> {

   public int compare(Integer x, Integer y) {

      return y.compareTo(x);

 }
}
```

Main method of Comparator ADT

- compare(x, y): returns an integer i such that
  - $i < 0$ if $x < y$
  - $i = 0$ if $x = y$
  - $i > 0$ if $x > y$
  - Error if x and y cannot be compared
- A generic priority queue uses an external comparator
  - the same keys can be prioritized using different comparators
- A comparator encapsulates the action of comparing two objects according to a given total order relation

Total order relations

- For a comparison rule $\leq$, it must satisfy the following properties for any keys k_1, k_2, k_3
  - Comparability: $k_1 \leq k_2$ or $k_2 \leq k_1$
  - Antisymmetric: if $k_1 \leq k_2$ and $k_2 \leq k_1$, then $k_1 = k_2$
  - Transitive: if $k_1 \leq k_2$ and $k_2 \leq k_3$, then $k_1 \leq k_3$

- If a set of elements has a total order defined for it, then the notion of minimal key, $k_{\min}$, is well defined
  - $k_{\min} \leq k$, for any other k

Examples for total order relationships

- Alphabetical order (or reserve alphabetical order)
  - $a \leq b$ if a is before b in alphabetical order
- Length of strings
  - $x \leq y$ if length of string x is less or equal to string y
- Lexicographical order with strings
  - Compare the string character by character, and the first element that differs determines the order
  - If all compared characters are equal, the short string ≤ long string
- $\leq$ or $\geq$ for numbers is a total order
- Order of pairs
  - For coordinate pairs $p = (x_1, y_1)$ and $q = (x_2, y_2)$, define $p \leq q$ if

$$x_1 < x_2, \text{ or } x_1 = x_2 \text{ and } y_1 \leq y_2 \quad \text{(Lexicographical order)}$$

Examples for comparator

```
public class Lexicographic  implements
Comparator<Point2D> {
  /*nested 2D point class*/
  int x1, y1, x2, y2;
  public int compare(Point2D a, Point2D b){
        x1 = a.getX();
        y1 = a.getY();
        x2 = b.getX();
        y2 = b.getY();
   If (x1 != x2)
      return (x1 – x2);
   else
      return (y1 – y2);
   }
 }
```

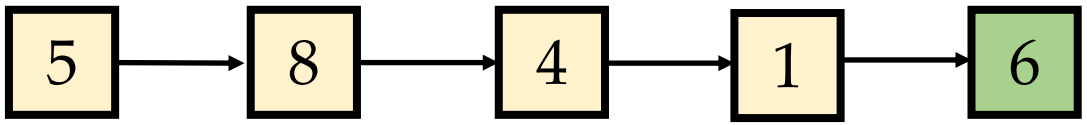
```
public class Point2D {
  protected int x, y;
  public Point2D(int x1, int y1){
    x = x1;
    y = y1;
  }
   public int getX(){
     return x;
  }
   public int getY(){
     return y;
   }
}
```

Priority Queue and Heap: roadmap

- Priority queue ADT
- Implementing priority queues using lists
  - Unsorted list
  - Sorted list
- Heap
  - Insert and delete methods
  - Heap construction
- Heap sort

Implementing priority queue using unsorted list

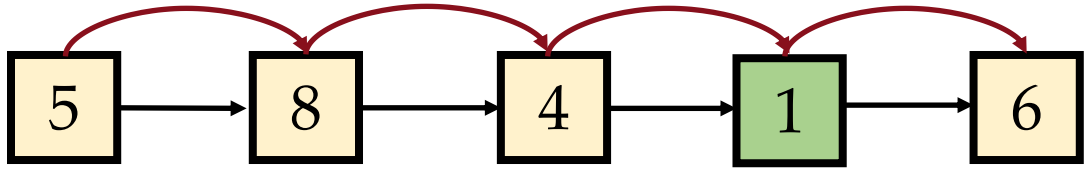
```
list ← new positional list
Algorithm insert(k,v)
  newEntry ← new PQEntry(k,v)
  list.addLast(newEntry)
```



- Time efficiency: $O(1)$

Implementing priority queue using unsorted list

```
list ← new positional list
Algorithm removeMin()
  current ← list.first()   ▷ position
  p ← list.first()           of min key
  If list.isEmpty() then
      return null
  Else
    While p != null do
      if p.key ≤ current.key do
          current ← p
      p ← list.after(p)
    return list.remove(current)
```



- Time efficiency: $O(n)$

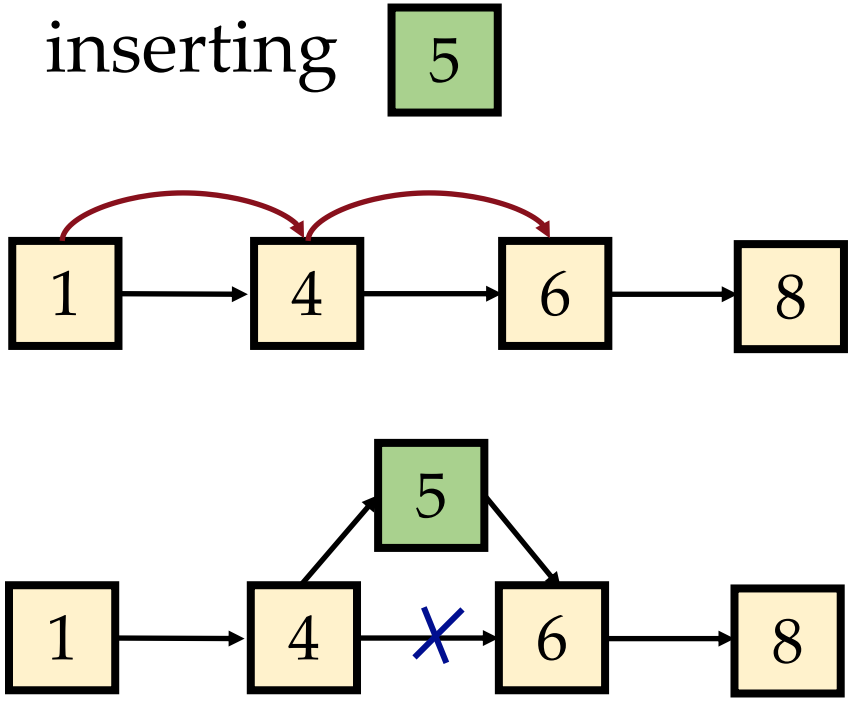
Observation: removeMin() always takes $O(n)$ even though the list is already sorted

Priority Queue and Heap: roadmap

- Priority queue ADT
- Implementing priority queues using lists
  - Unsorted list
  - Sorted list
- Heap
  - Insert and delete methods
  - Heap construction
- Heap sort

Implementing priority queue using sorted list

```
list ← new positional list
Algorithm insert(k,v)
  newEntry ← new PQEntry(k,v)
  p ← list.first()
  While p != null and p.key ≤ k do
      p ← list.after(p)
  If p = null then
      list.addLast(newEntry)
  Else
      list.addBefore(p, newEntry)
```

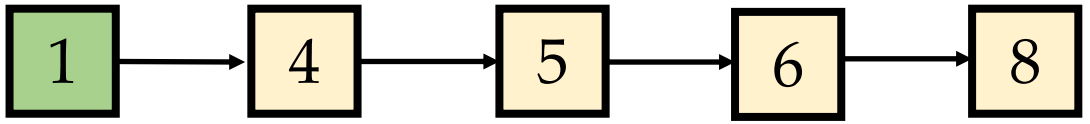


- Time efficiency: $O(n)$

Observation: insert() takes $O(n)$ in the worst case.

Implementing priority queue using sorted list

```
list ← new positional list
Algorithm removeMin()
  If list.isEmpty() then
      return null
  Else
      return list.remove(list.first())
```



- Time efficiency: $O(1)$

Performance comparisons

PQ using unsorted list

insert()	**O(1)**
min()	**Θ(n)**
removeMin()	**Θ(n)**

PQ using sorted list

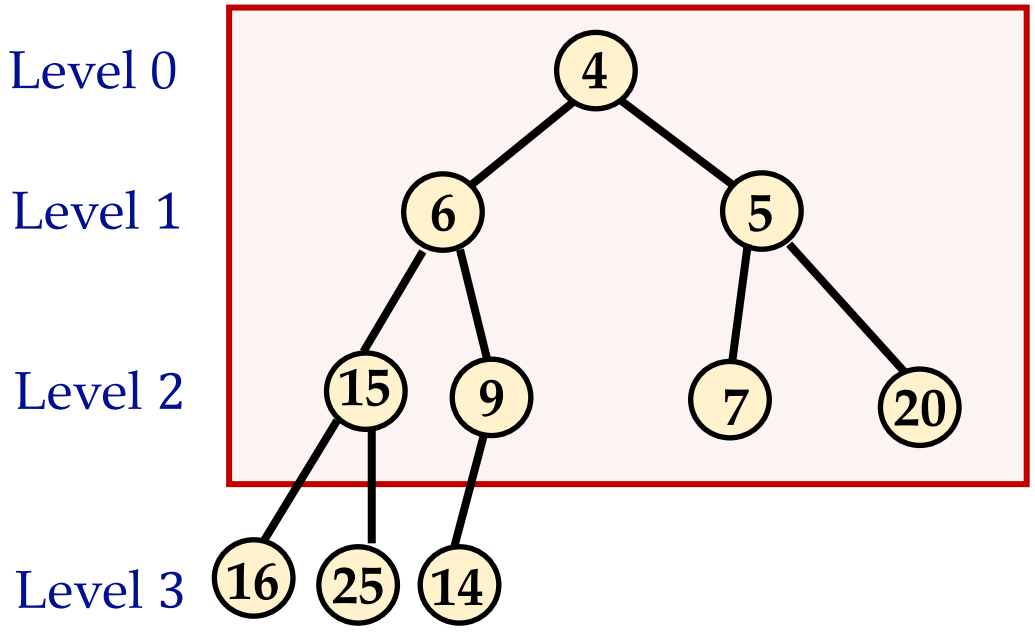
insert()	**O(n)**
min()	**O(1)**
removeMin()	**O(1)**

Priority Queue and Heap: roadmap

- Priority queue ADT
- Implementing priority queues using lists
  - Unsorted list
  - Sorted list
- Heap
  - Insert and delete methods
  - Heap construction
- Heap sort

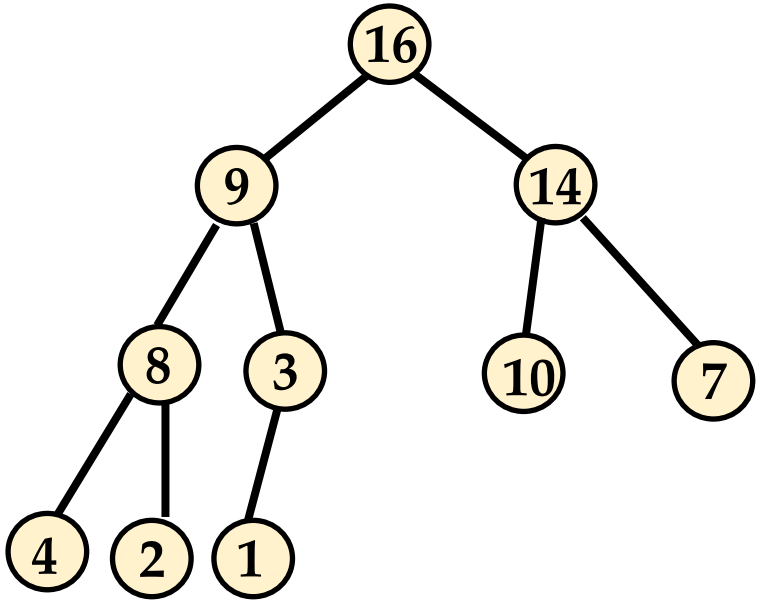
Heap (Min-heap)

- A min-heap is a complete binary tree that stores keys or key-value pairs and satisfies
  - heap-order property: key(parent) ≤ key(child)



Min-heap

key(parent) ≤ key(child)

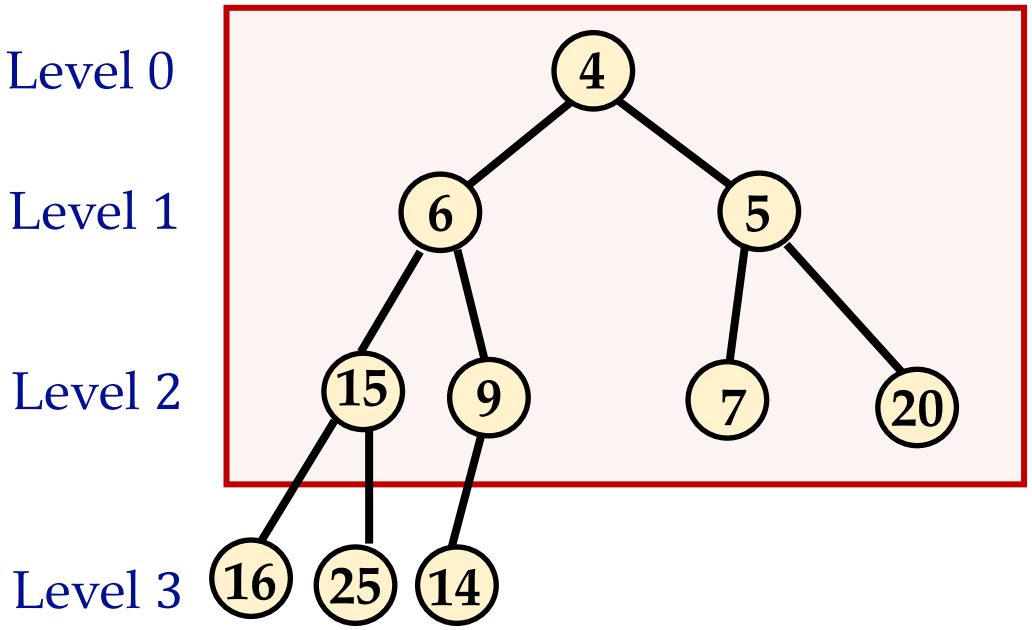


Max-heap

key(parent) ≥ key(child)

Height of a heap

- A heap that stores n keys has height $\lfloor \log n \rfloor \in \Theta(\log n)$

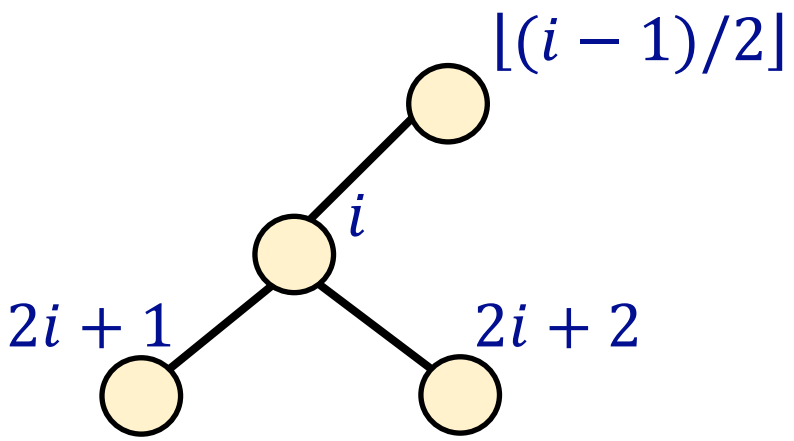


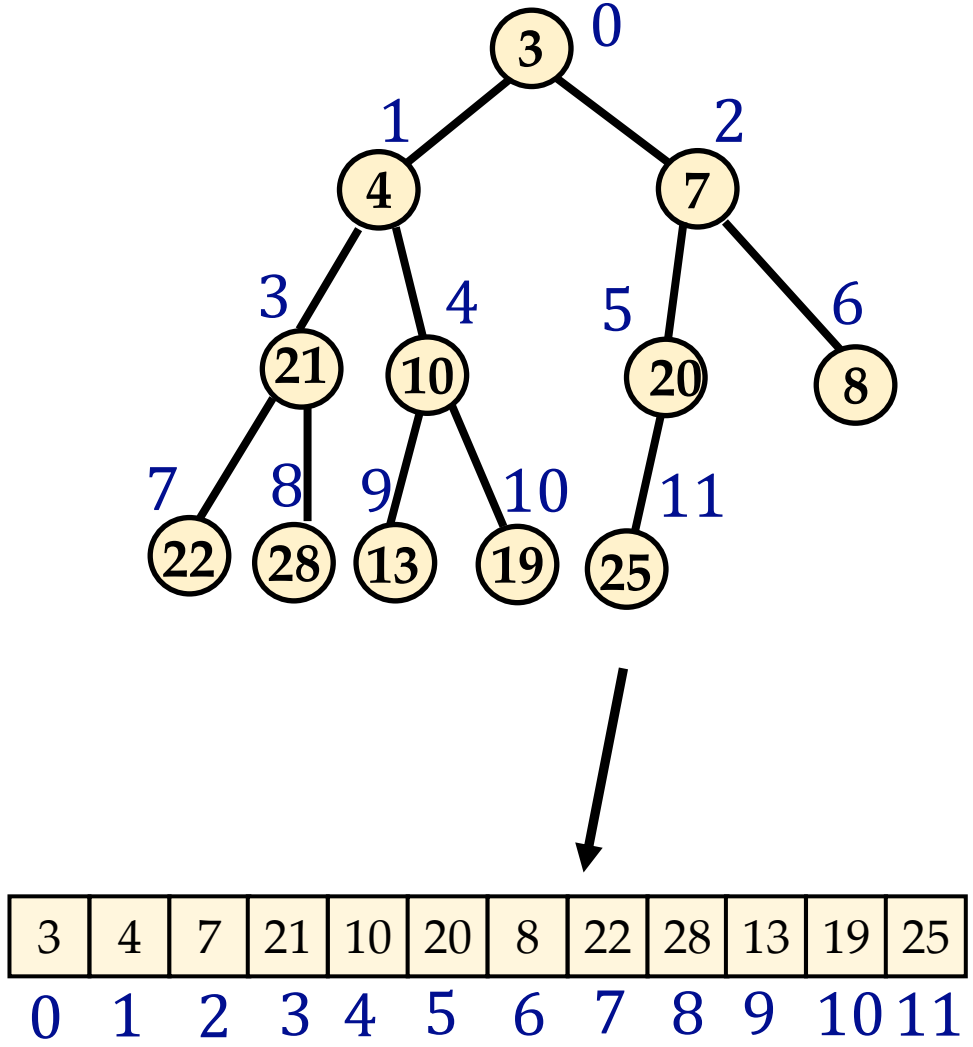
Recall the height of a complete binary tree is

$$h = \lfloor \log n \rfloor$$

Heap implemented by array

- Heap can be nicely implemented by an array, where the node with index i has
  - Left child at index $2i + 1$
  - Right child at index $2i + 2$
  - Parent at index $\lfloor (i - 1)/2 \rfloor$



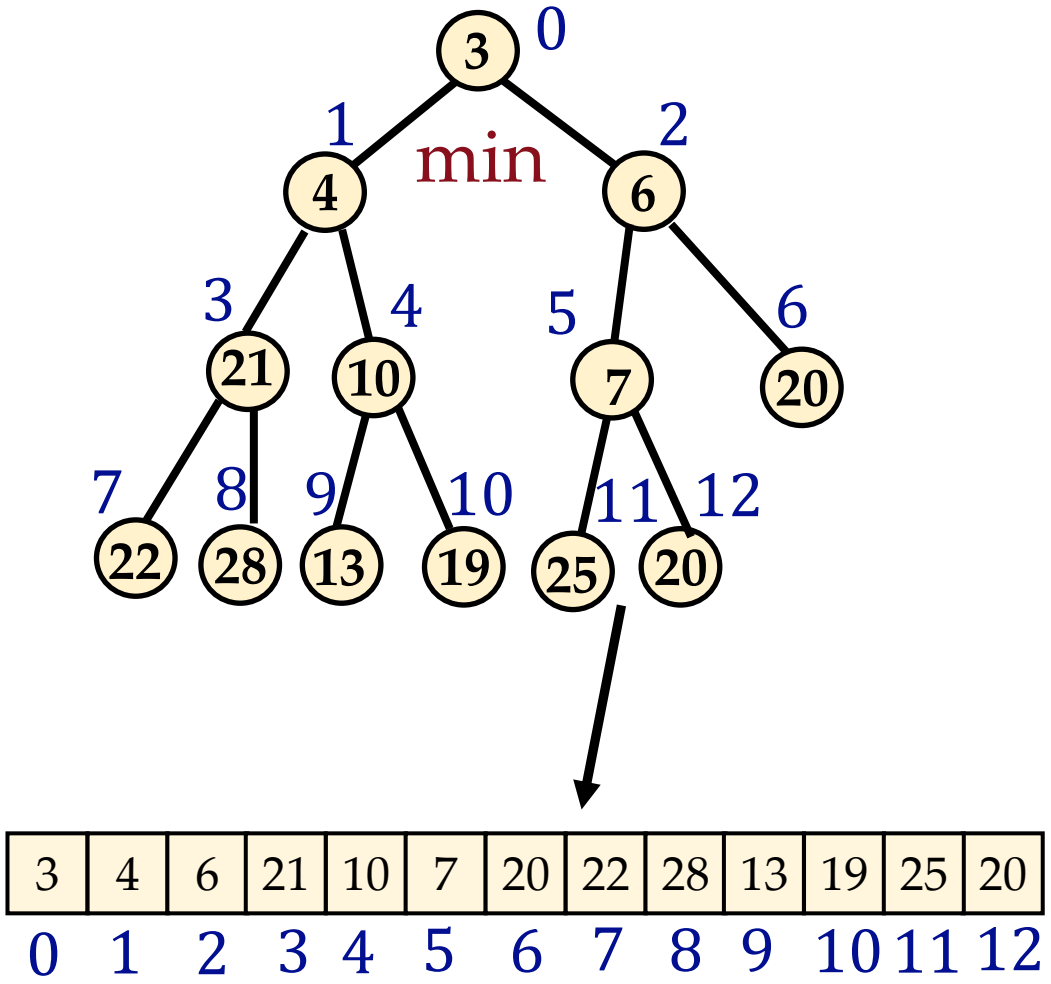


Priority Queue and Heap: roadmap

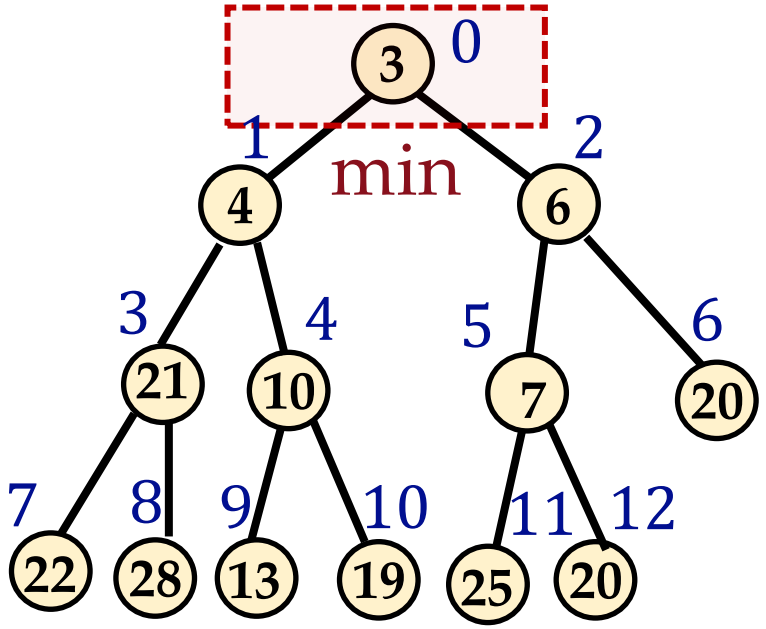
- Priority queue ADT
- Implementing priority queues using lists
  - Unsorted list
  - Sorted list
- Heap
  - Insert and delete methods
  - Heap construction
- Heap sort

Implementing a priority queue with a heap

```
heap ← new array
 Algorithm min()
   If heap.isEmpty() then
     return null
   Else
     return heap.get(0)
```



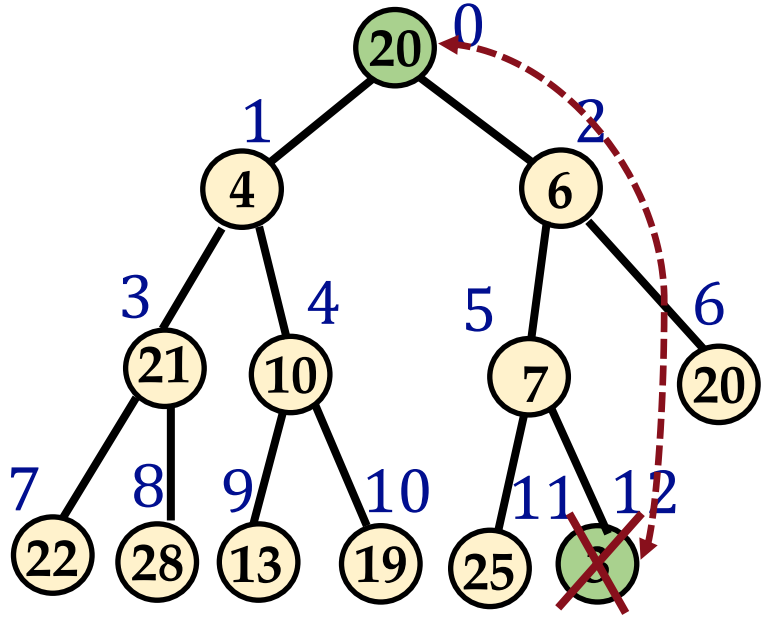
Heap removal



- Swap root and the last node and then remove the last node
  - To maintain a complete binary tree

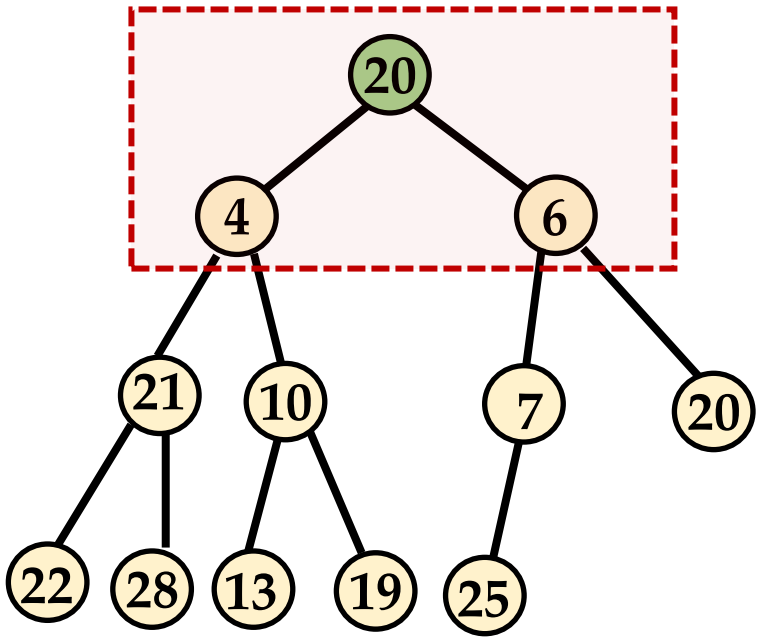
Heap removal

```
heap ← new array
Algorithm removeMin()
  If heap.isEmpty() then
    return null
  Else
    temp = heap.get(0)
    swap(0, heap.size() − 1)
    heap.remove(heap.size() − 1)
    downHeap(0)
    return temp
```



- Swap root and the last node and then remove the last node
  - To maintain a complete binary tree
- Down-heap bubbling:
  - To maintain the heap-order property

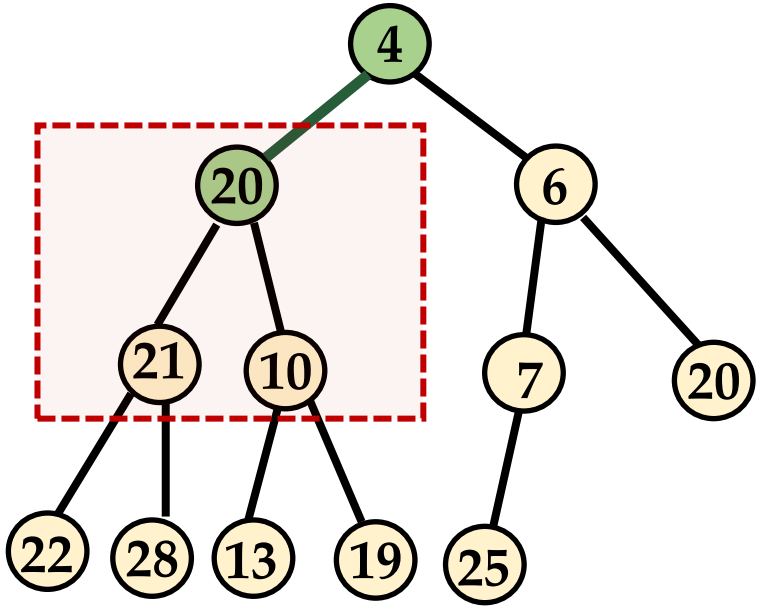
Down-heap bubbling



Why must we choose the smaller child?

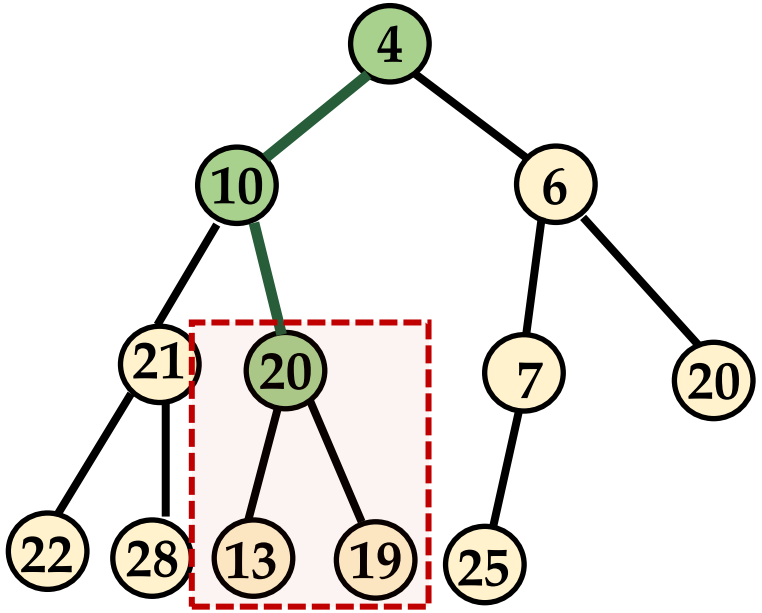
if key(parent) > min{key(leftChild), key(rightChild)},
swap key(parent) with the smaller key between key(leftChild) and key(rightChild)
to maintain heap-order property

Down-heap bubbling



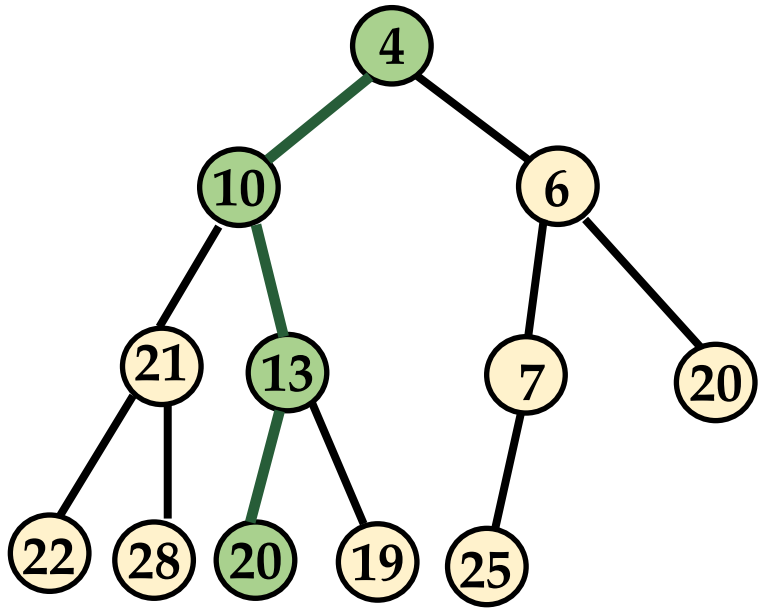
if key(parent) > min{key(leftChild), key(rightChild)},
swap key(parent) with the smaller key between key(leftChild) and key(rightChild)
to maintain heap-order property

Down-heap bubbling



if key(parent) > min{key(leftChild), key(rightChild)},
swap key(parent) with the smaller key between key(leftChild) and key(rightChild)
to maintain heap-order property

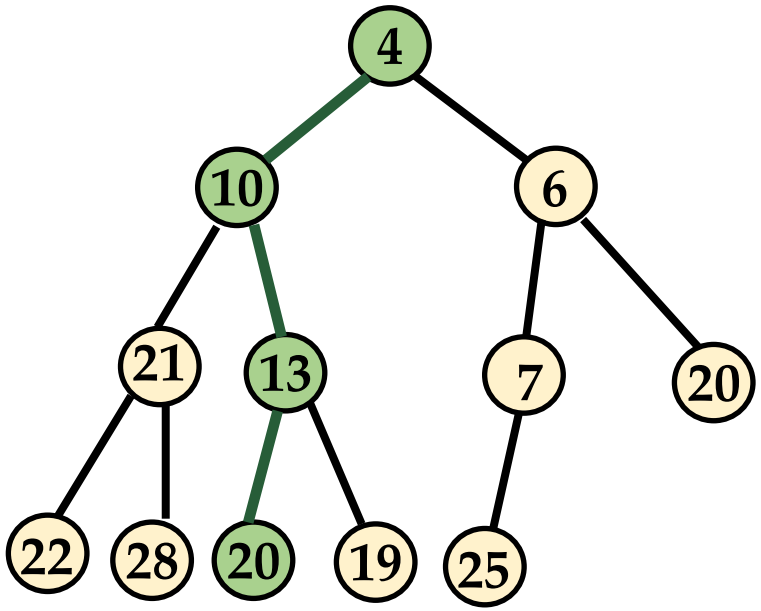
Down-heap bubbling




```
Algorithm downHeap(j)
  While hasLeft(j) then
    smallIndex ← left(j)
    If hasRight(j) and
    key(left(j))>key(right(j)) then
      smallIndex ← right(j)
    If key(smallIndex)>key(j) then
      return
    swap(smallIndex, j)
    j ← smallIndex
```

if key(parent) > min{key(leftChild), key(rightChild)},
swap key(parent) with the smaller key between key(leftChild) and key(rightChild)
to maintain heap-order property

Down-heap bubbling: performance



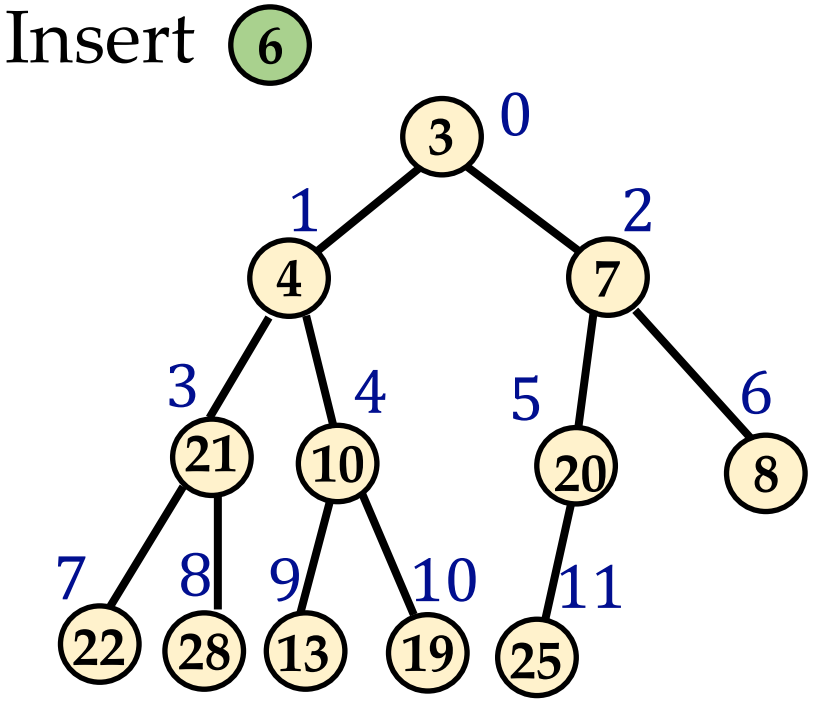

```
Algorithm downHeap(j)
  While hasLeft(j) then
    smallIndex ← left(j)
    If hasRight(j) and
    key(left(j))>key(right(j)) then
      smallIndex ← right(j)
    If key(smallIndex)>key(j) then
      return
    swap(smallIndex, j)
    j ← smallIndex
```

How many swaps in the worst case?

- Down-heap terminates when the key is no greater than the keys of both its children or the bottom of the heap is reached

$$\text{Total number of swaps} \leq h \in O(\log n)$$

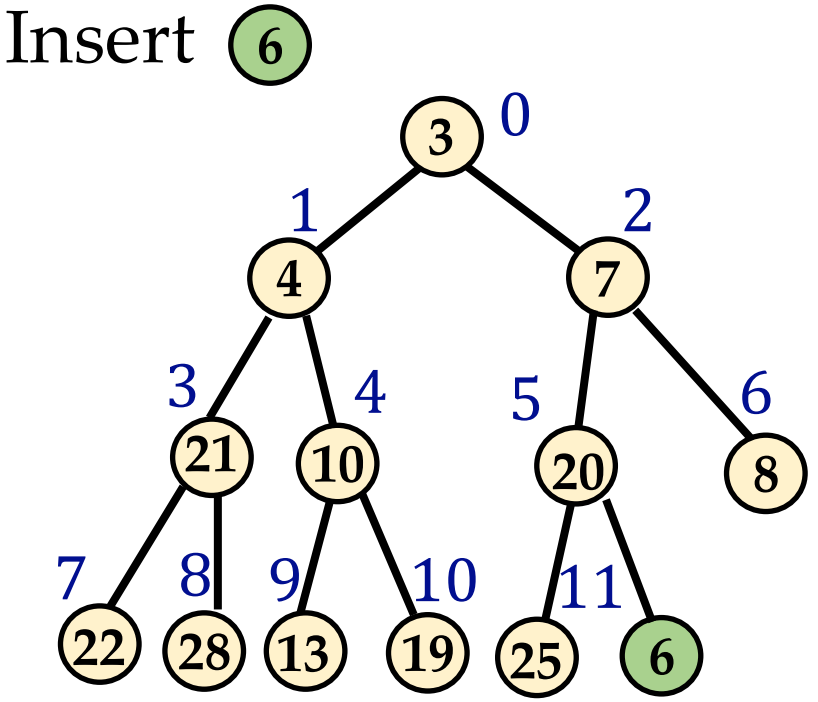
Heap insertion



- Add the key in the next available position in the heap
  - To maintain a complete binary tree

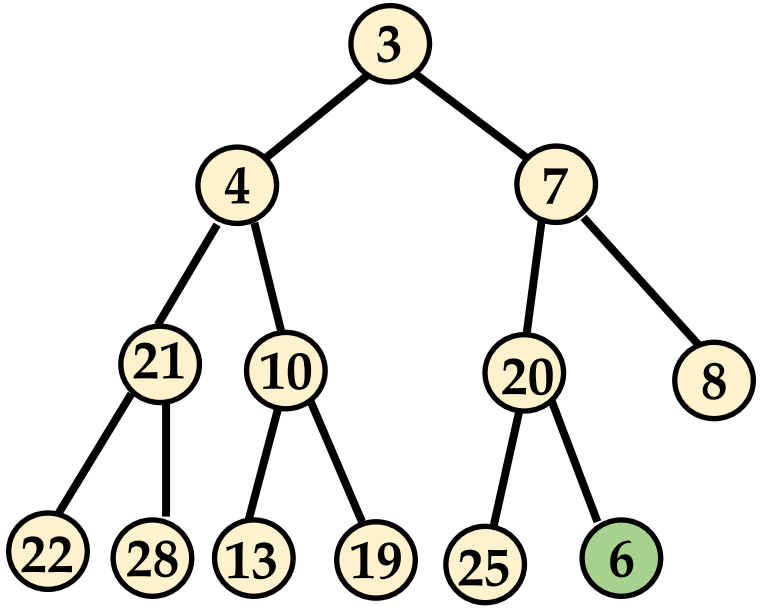
Heap insertion

```
Algorithm insert(k,v)
  newEntry ← new PQEntry(k,v)
  heap.addLast(newEntry)
  upHeap(heap.size() − 1)
```



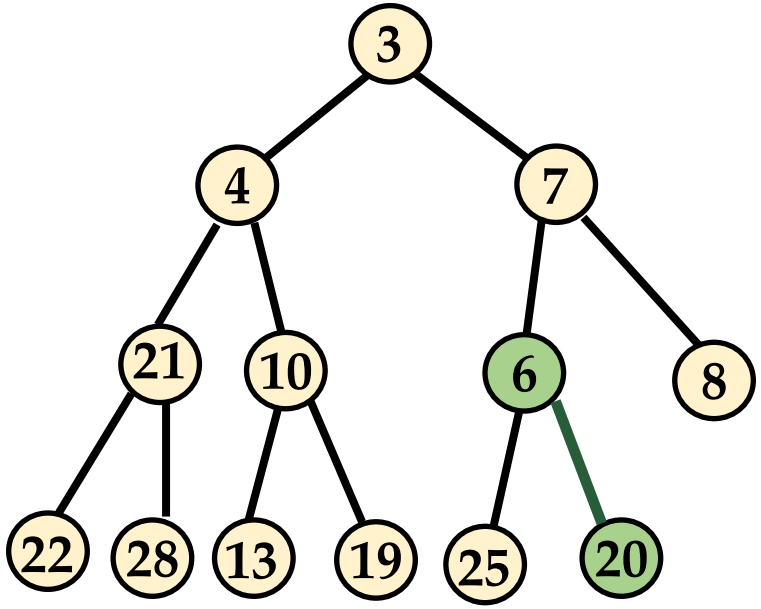
- Add the key in the next available position in the heap
  - To maintain a complete binary tree
- Up-heap bubbling
  - To maintain the heap-order property

Up-heap bubbling



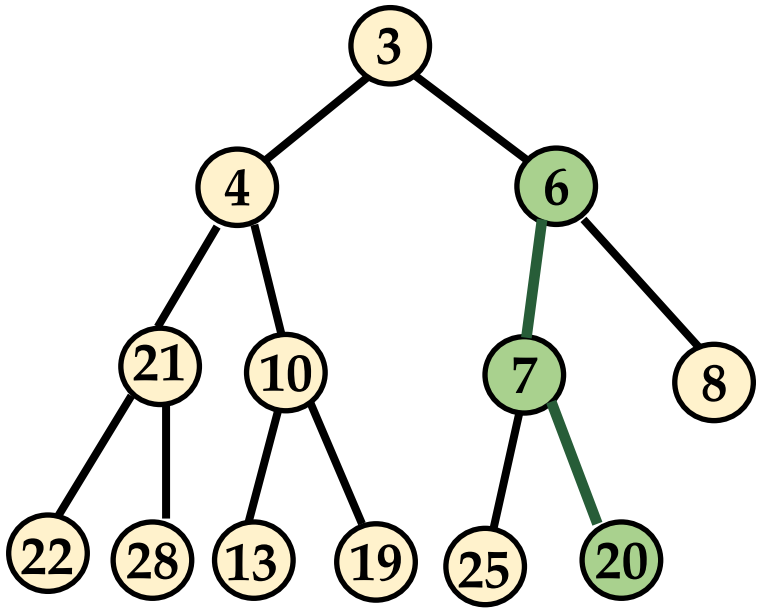
if key(parent) > key(Child),
swap key(parent) and key(Child)
to maintain heap-order property

Up-heap bubbling



if key(parent) > key(Child),
swap key(parent) and key(Child)
to maintain heap-order property

Up-heap bubbling




```
Algorithm upHeap(j)
  While j > 0 then
    If key(parent(j)) < key(j) then
      return
    swap(parent(j), j)
    j ← parent(j)
```

How many swaps in the worst case?

- Upheap terminates when new key is greater than the key of its parent or the top of the heap is reached

Total number of swaps $\leq h \in O(\log n)$

Performance comparisons

PQ using heap

Insert()	O(log n)
min()	O(1)
removeMin()	O(log n)

up-heap

Remove root + down-heap

PQ using unsorted list

insert()	O(1)
min()	O(n)
removeMin()	O(n)

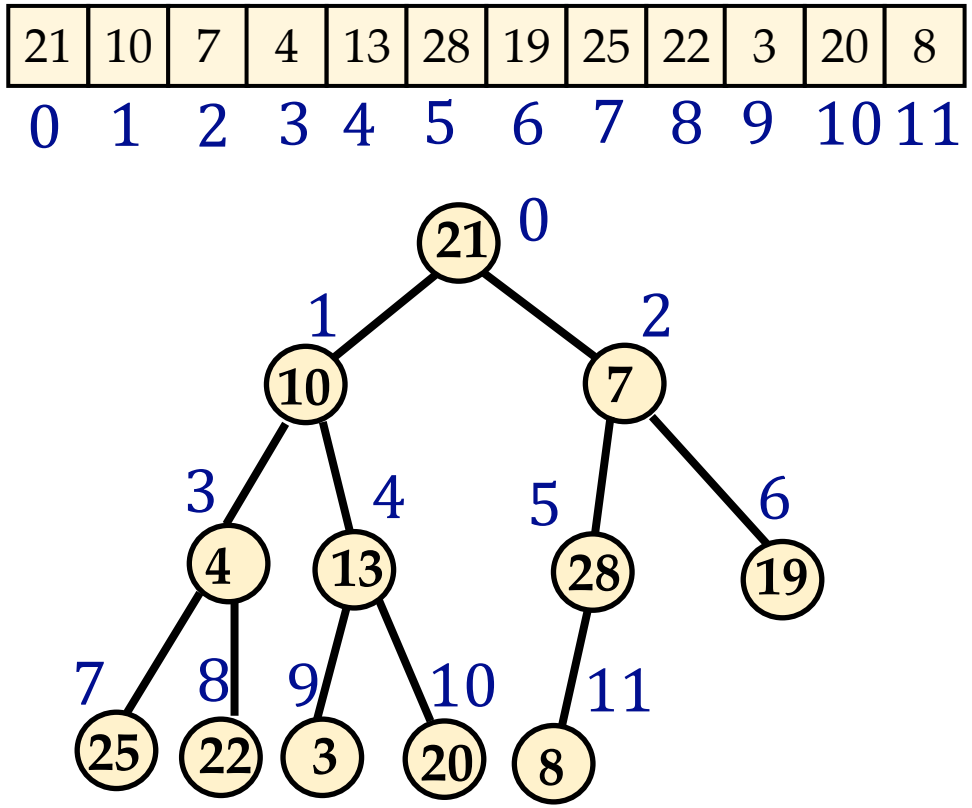
PQ using sorted list

insert()	O(n)
min()	O(1)
removeMin()	O(1)

Priority Queue and Heap: roadmap

- Priority queue ADT
- Implementing priority queues using lists
  - Unsorted list
  - Sorted list
- Heap
  - Insert and delete methods
  - Heap construction
- Heap sort

Heap construction



- Given an array of n entries, heapify it

- Intuitive approach: insert the entries one at a time using a series of heap insertions

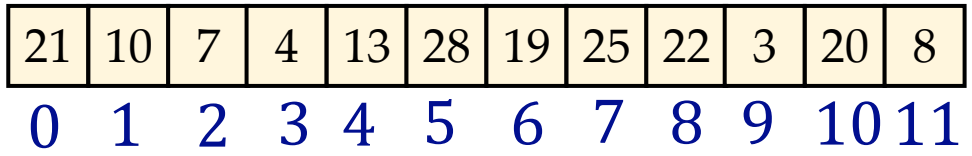
Total number of swaps $\leq \sum_{k=1}^{n} \log k$

$\leq \sum_{k=1}^{n} \log n$

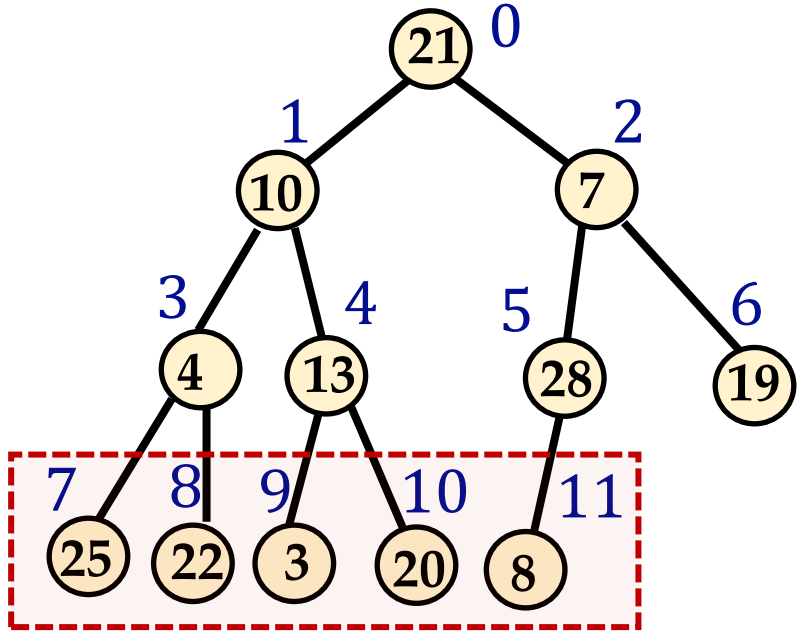
$\in O(n \log n)$

- But, we can actually build a heap in $O(n)$!

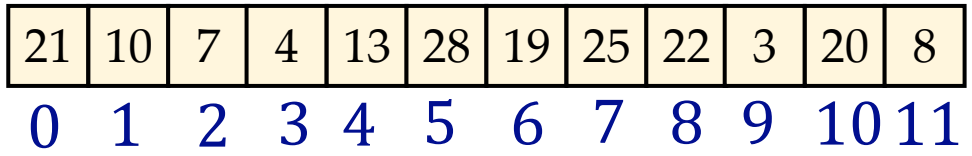
Bottom-up heap construction



- Consider the nodes at the bottom level. Each of them is a heap

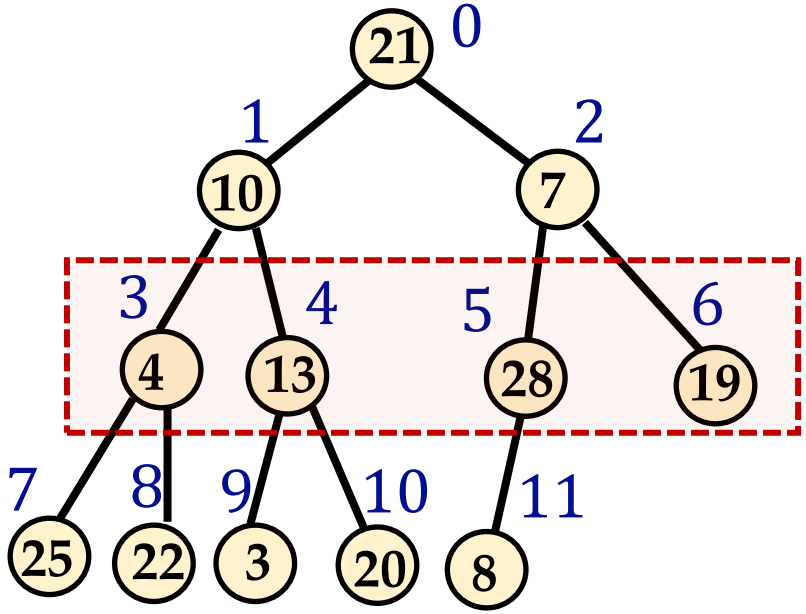


Bottom-up heap construction

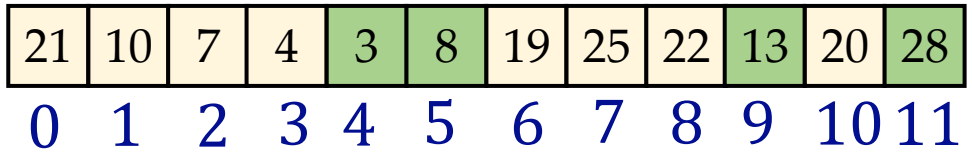


- Consider the nodes at the bottom level. Each of them is a heap

- Consider the nodes on the second-to-last level:
  - the subtrees rooted at these nodes are almost-heap (only the root might violate the heap-order property).
  - Re-arrange to heapify the subtrees by down-heap

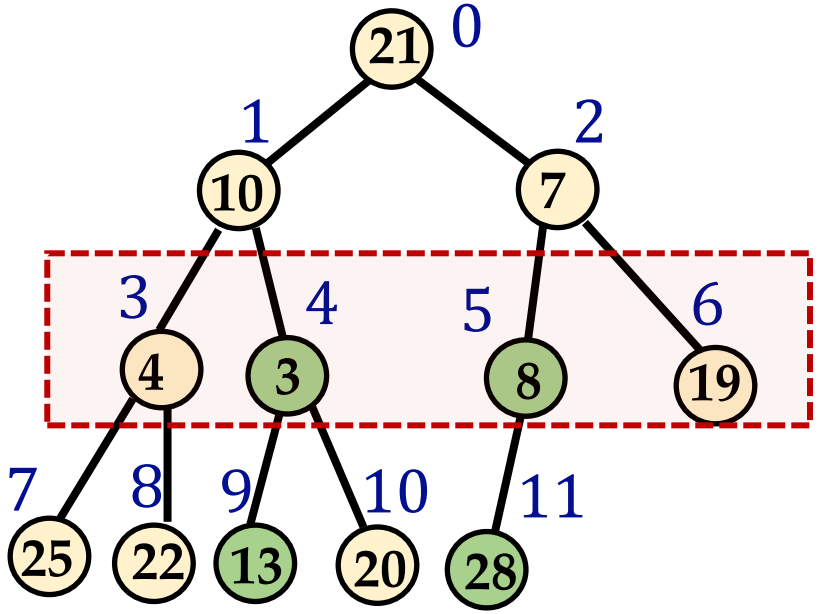


Bottom-up heap construction

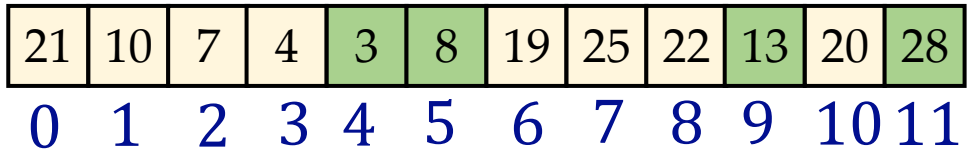


- Consider the nodes at the bottom level. Each of them is a heap

- Consider the nodes on the second-to-last level:
  - the subtrees rooted at these nodes are almost-heap (only the root might violate the heap-order property).
  - Re-arrange to heapify the subtrees by down-heap



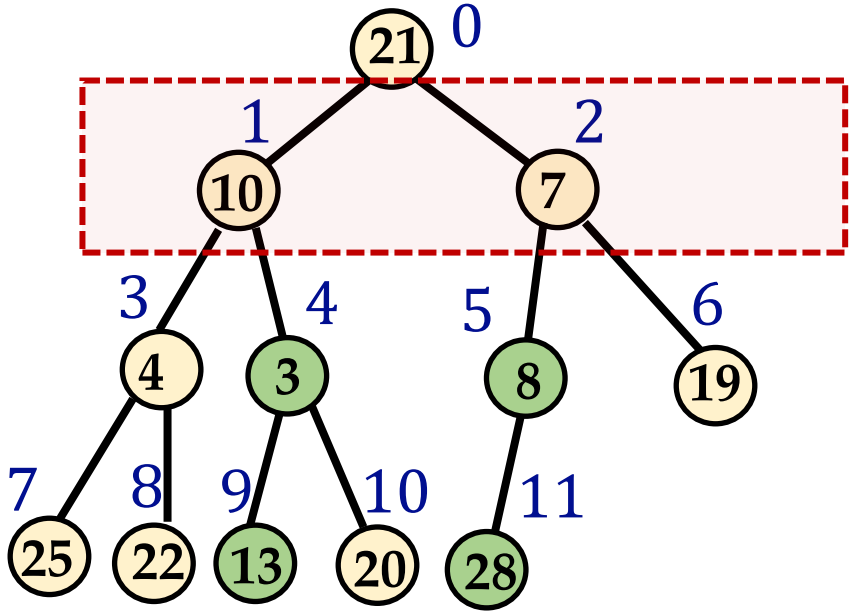
Bottom-up heap construction



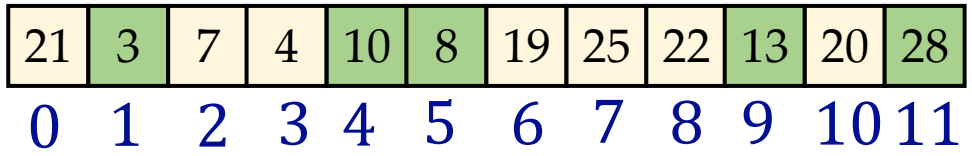
- Consider the nodes at the bottom level. Each of them is a heap

- Consider the nodes on the second-to-last level:
  - the subtrees rooted at these nodes are almost-heap (only the root might violate the heap-order property).
  - Re-arrange to heapify the subtrees by down-heap

- Consider the nodes on upper levels until level 0



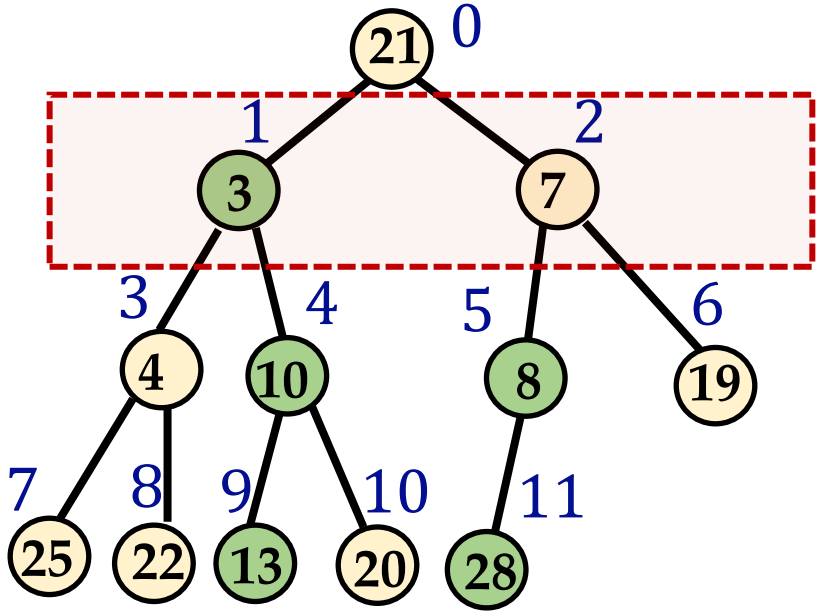
Bottom-up heap construction



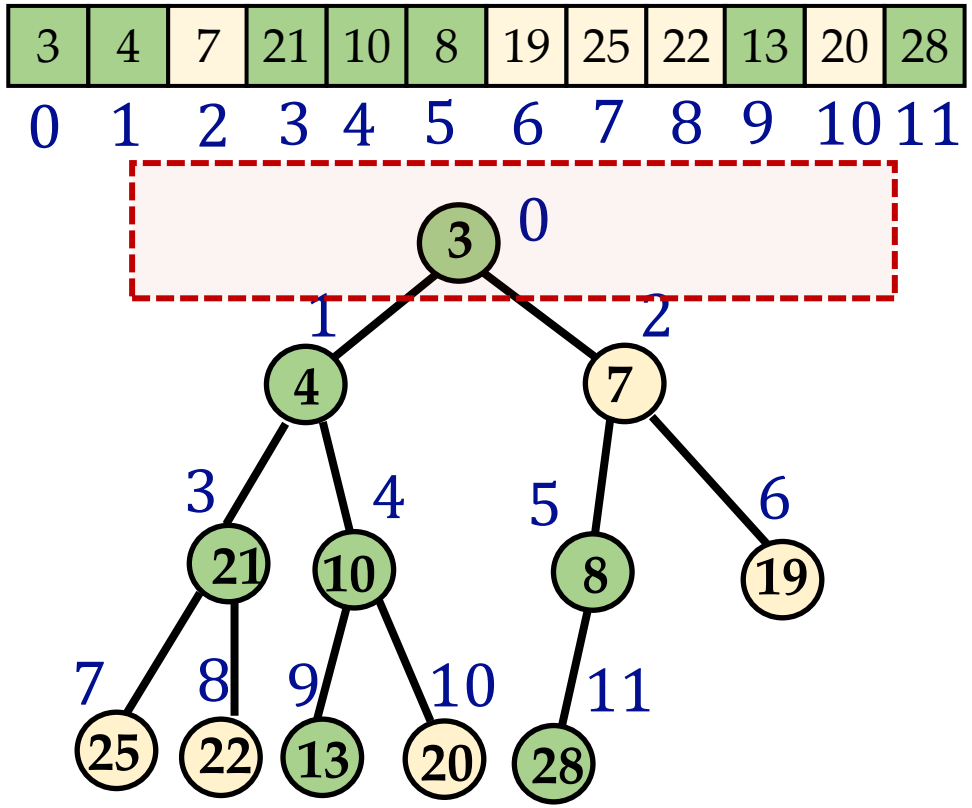
- Consider the nodes at the bottom level. Each of them is a heap

- Consider the nodes on the second-to-last level:
  - the subtrees rooted at these nodes are almost-heap (only the root might violate the heap-order property).
  - Re-arrange to heapify the subtrees by down-heap

- Consider the nodes on upper levels until level 0

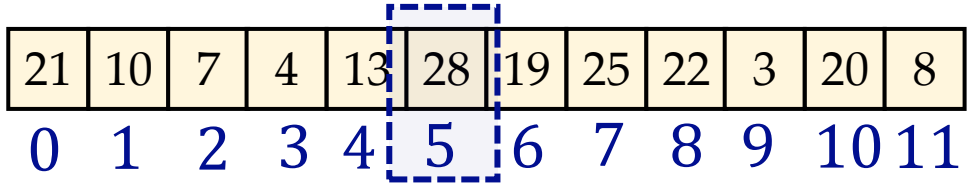


Bottom-up heap construction



- Consider the nodes at the bottom level. Each of them is a heap
- Consider the nodes on the second-to-last level:
  - the subtrees rooted at these nodes are almost-heap (only the root might violate the heap-order property).
  - Re-arrange to heapify the subtrees by down-heap
- Consider the nodes on upper levels until level 0

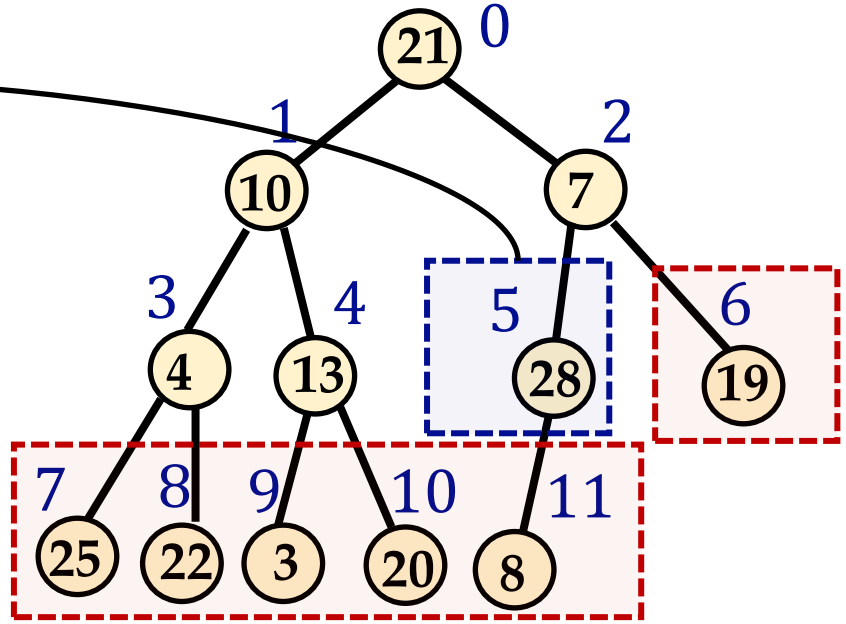
Bottom-up heap construction



Algorithm buildHeap(A)

For i $\leftarrow \left\lfloor \frac{A.size()}{2} \right\rfloor - 1$ **down to 0 then**

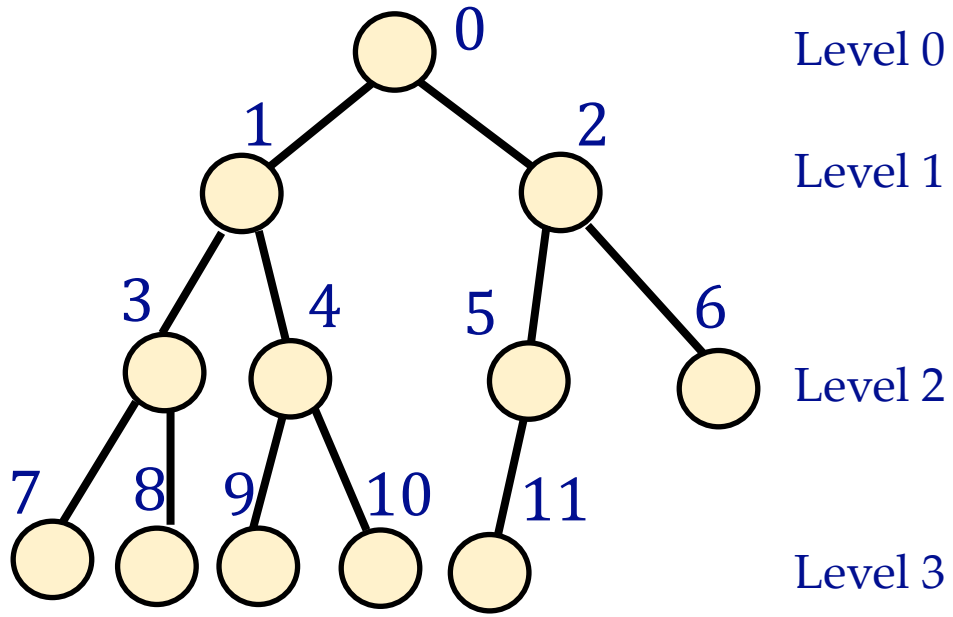
downHeap(i)



Bottom-up heap construction: a counting example

- Each down-heap call can move at most the height of its starting subtree

Level	Nodes	Max swaps per node	Subtotal
0	1	3	1 × 3 = 3
1	2	2	2 × 2 = 4
2	4	1	3 × 1 = 3
3	5	0	5 × 0 = 0



Keys are omitted for counting worst-case number of swaps

Total swaps ≤ 5 × 0 + 3 × 1 + 2 × 2 + 1 × 3 = 10

Total swaps of sequential insertion construction $\leq \sum_{k=1}^{12} \log k \approx 29$

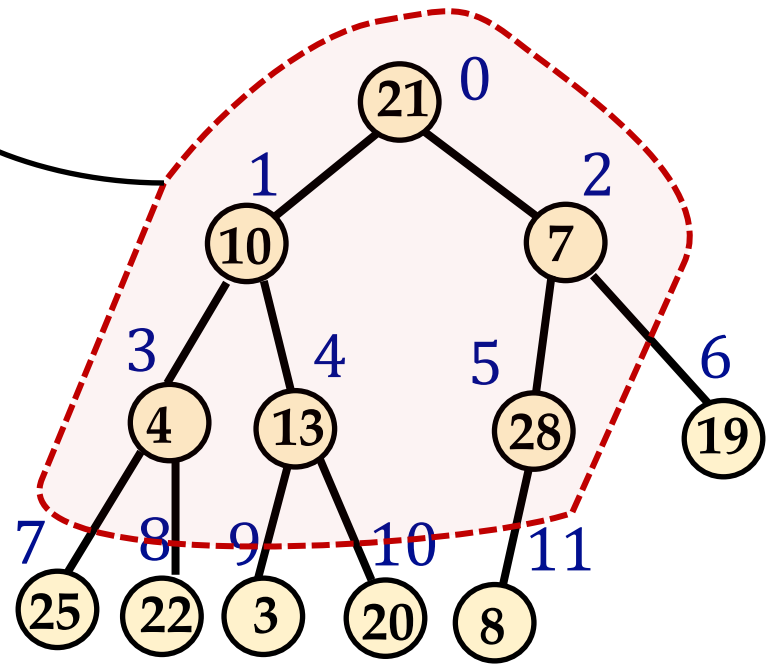
Most nodes can move only a little; only a few nodes can move far.

Bottom-up heap construction: Performance

Worst-case RT of down-heap

Total number of swaps $\leq \frac{n}{2} \times \log n$

$\in O(n \log n)$



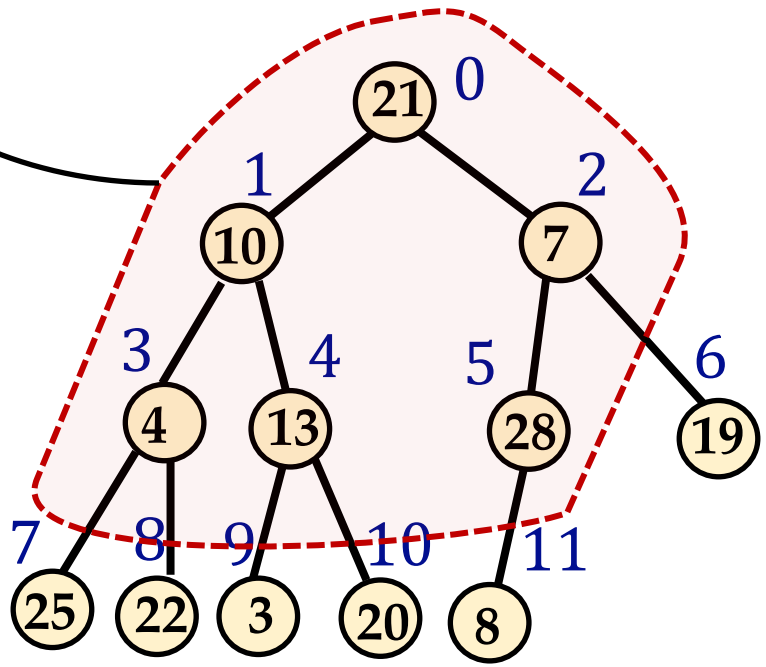
Bottom-up heap construction: Performance

Worst-case RT of down-heap

Total number of swaps $\leq \frac{n}{2} \times \log n$

$\in O(n \log n)$

The number of swaps for each down-heap depends on the height of the subtree rooted at each node



$O(n \log n)$ is not tight!

Bottom-up heap construction: Performance

Number of swaps of each node at level i

Number of nodes at level i

$$\text{Total swaps} \le \sum_{i=0}^{h-1} (h-i) \cdot 2^i$$

$$= \sum_{j=1}^{h} j \cdot 2^{h-j} \quad \text{(By letting } j = h - i\text{)}$$

$$= 2^h \cdot \sum_{j=1}^{h} j \cdot 2^{-j} \le 2^h \cdot 2 \in \boldsymbol{O(n)}$$

$$\sum_{j=1}^{h} j \cdot 2^{-j} = 1 \cdot 2^{-1} + 2 \cdot 2^{-2} + 3 \cdot 2^{-3} + \cdots + h \cdot 2^{-h}$$

$$= 1 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} + \cdots + 1 \cdot 2^{-h} \quad (\le 1)$$

$$+ 1 \cdot 2^{-2} + 1 \cdot 2^{-3} + \cdots + 1 \cdot 2^{-h} \quad (\le 2^{-1})$$

$$+ 1 \cdot 2^{-3} + \cdots + 1 \cdot 2^{-h} \quad (\le 2^{-2})$$

$$\cdots$$

$$+ 1 \cdot 2^{-h} \quad (\le 2^{-h+1})$$

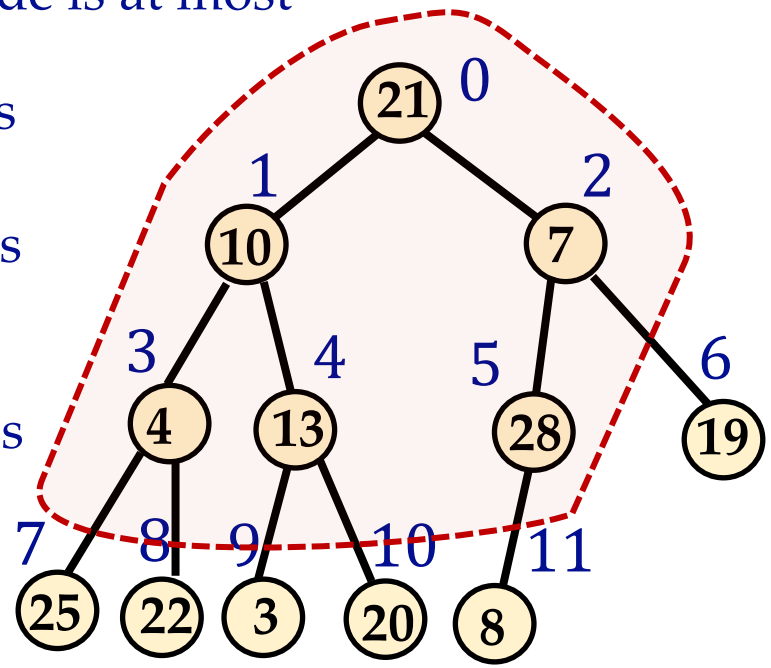
$$\le 1 + 2^{-1} + 2^{-2} + \cdots + 2^{-h+1} \le 2$$

The number of swaps for each node is at most

3 swaps

2 swaps

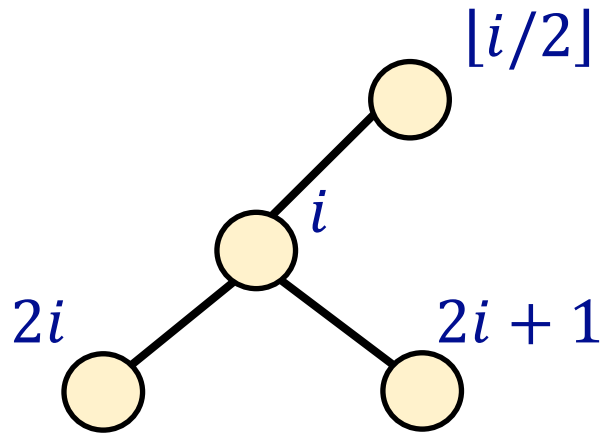
1 swaps

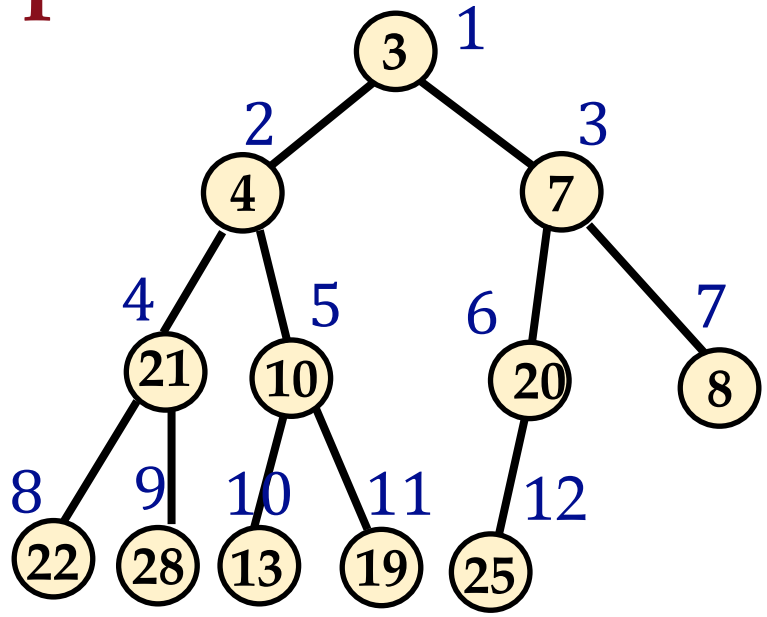


At most $h - i$ swaps for each node at level i

One implementation detail for heaps

- We can also number the nodes starting from 1. Then for each node with index i,
  - Left child at index $2i$
  - Right child at index $2i + 1$
  - Parent at index $\lfloor i/2 \rfloor$

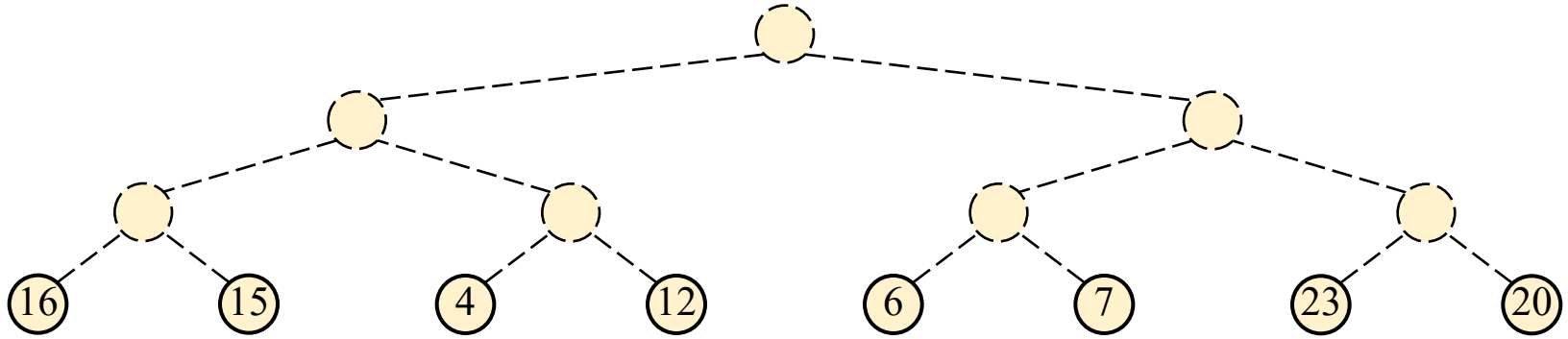


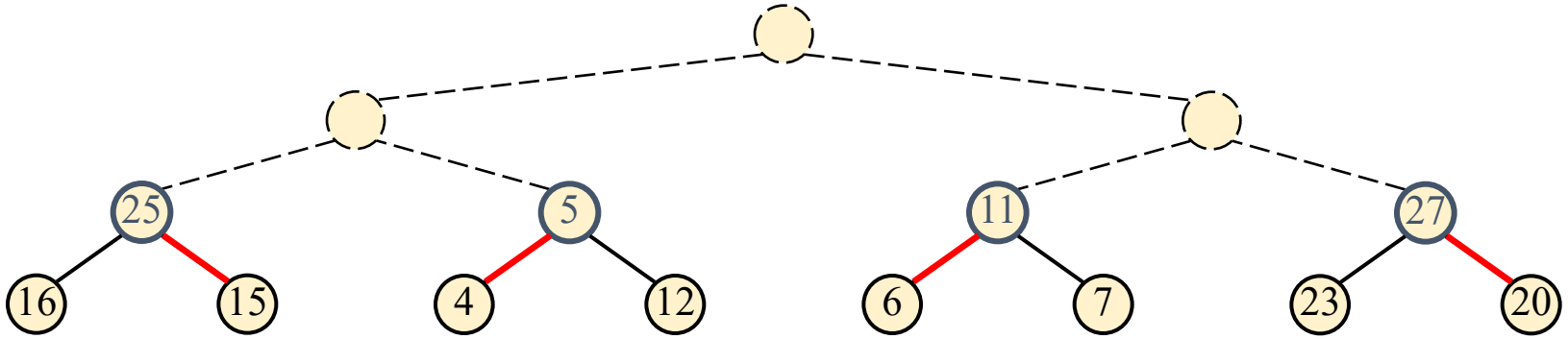


∅	3	4	7	21	10	20	8	22	28	13	19	25
0	1	2	3	4	5	6	7	8	9	10	11	12

Exercise

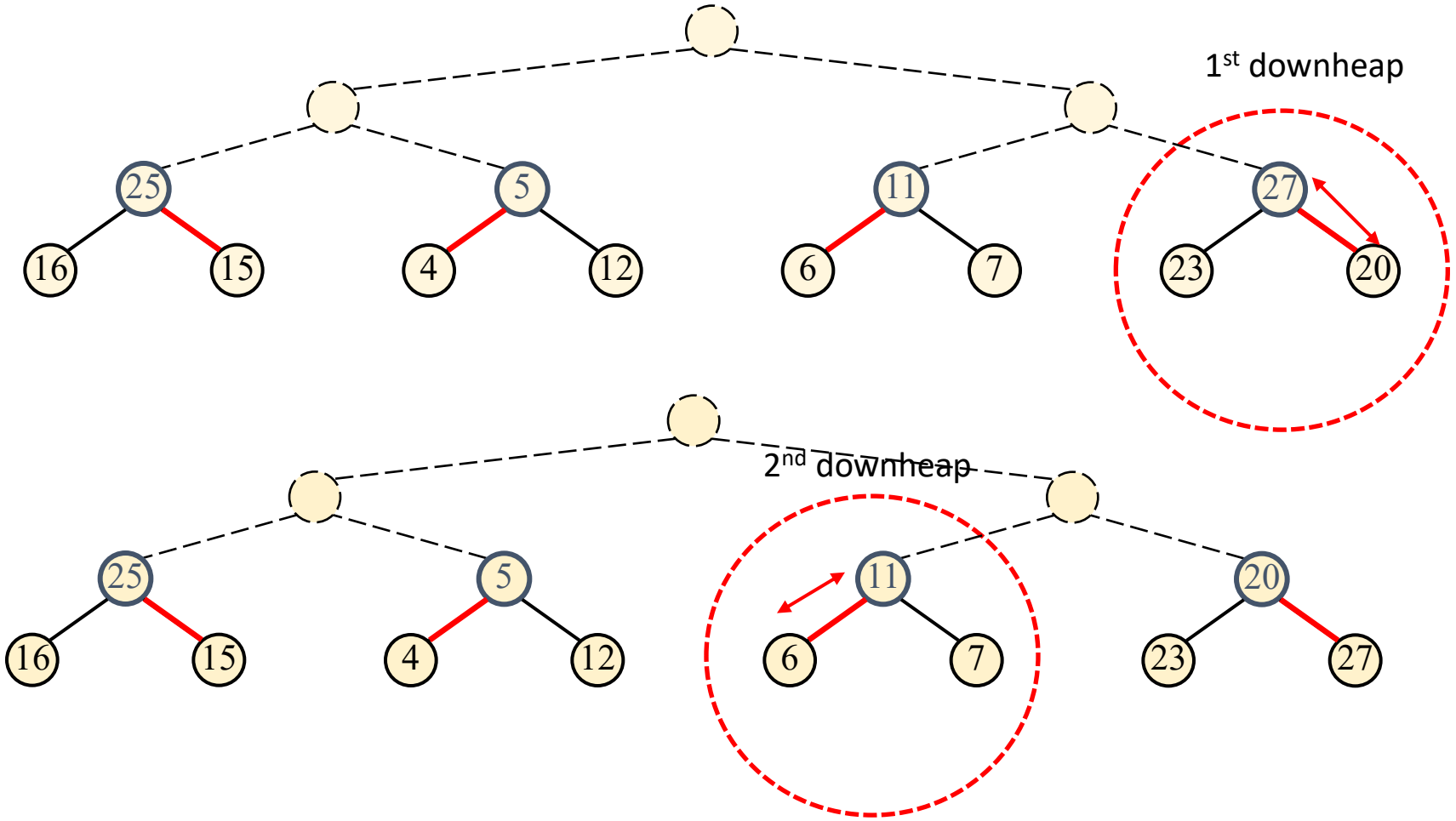
Construct a min-Heap using {14,9,8,25,5,11,27,16,15,4,12,6,7,23,20}





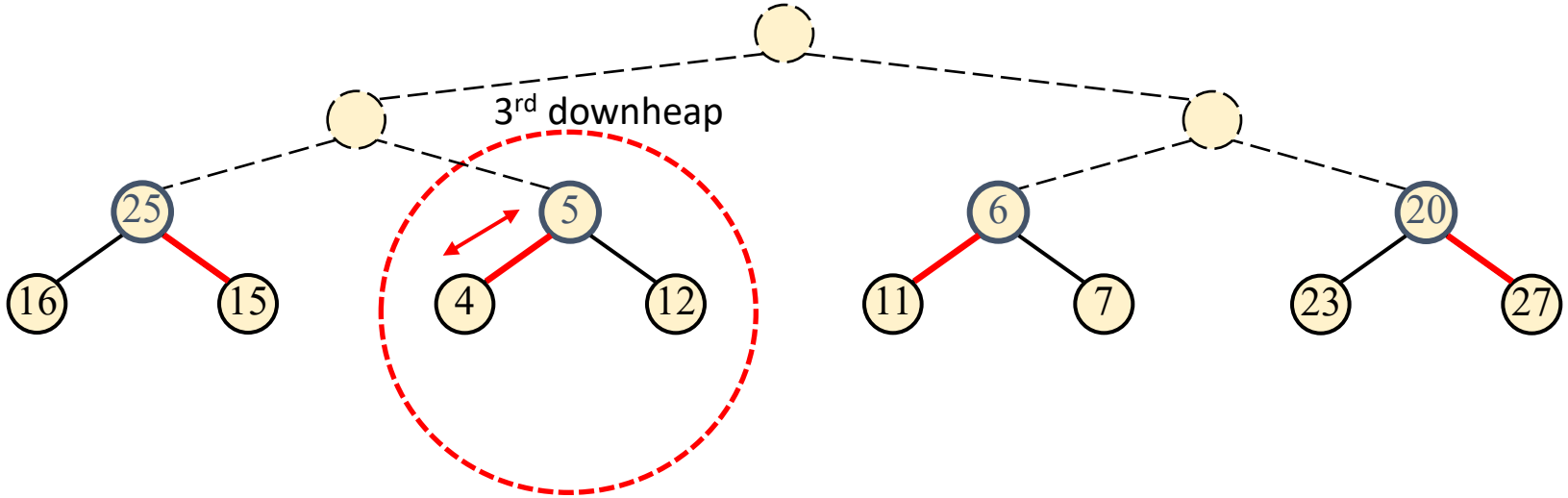
Exercise

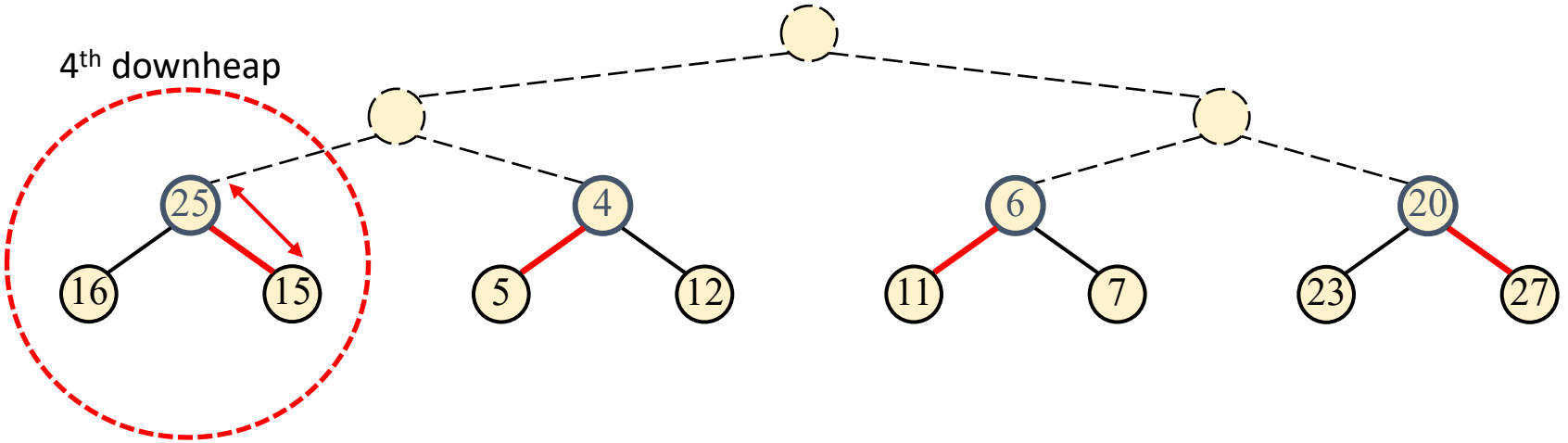
Construct a min-Heap using {14,9,8,25,5,11,27,16,15,4,12,6,7,23,20}



Exercise

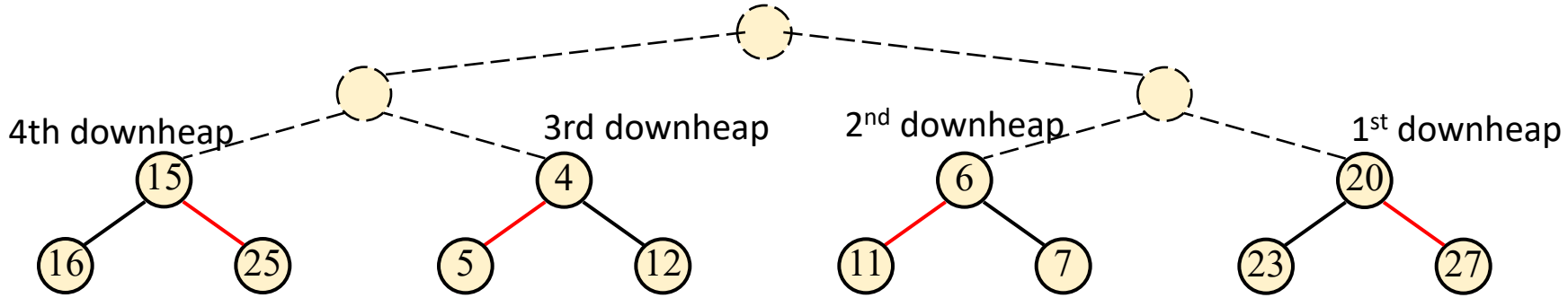
Construct a min-Heap using {14,9,8,25,5,11,27,16,15,4,12,6,7,23,20}

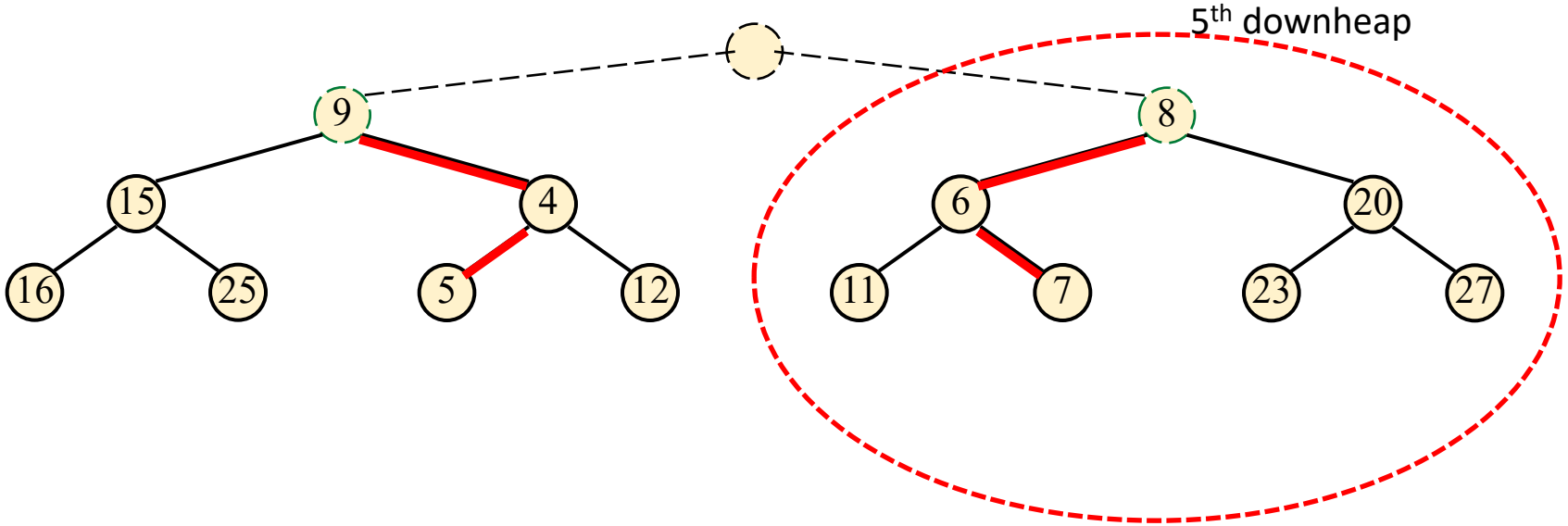




Exercise

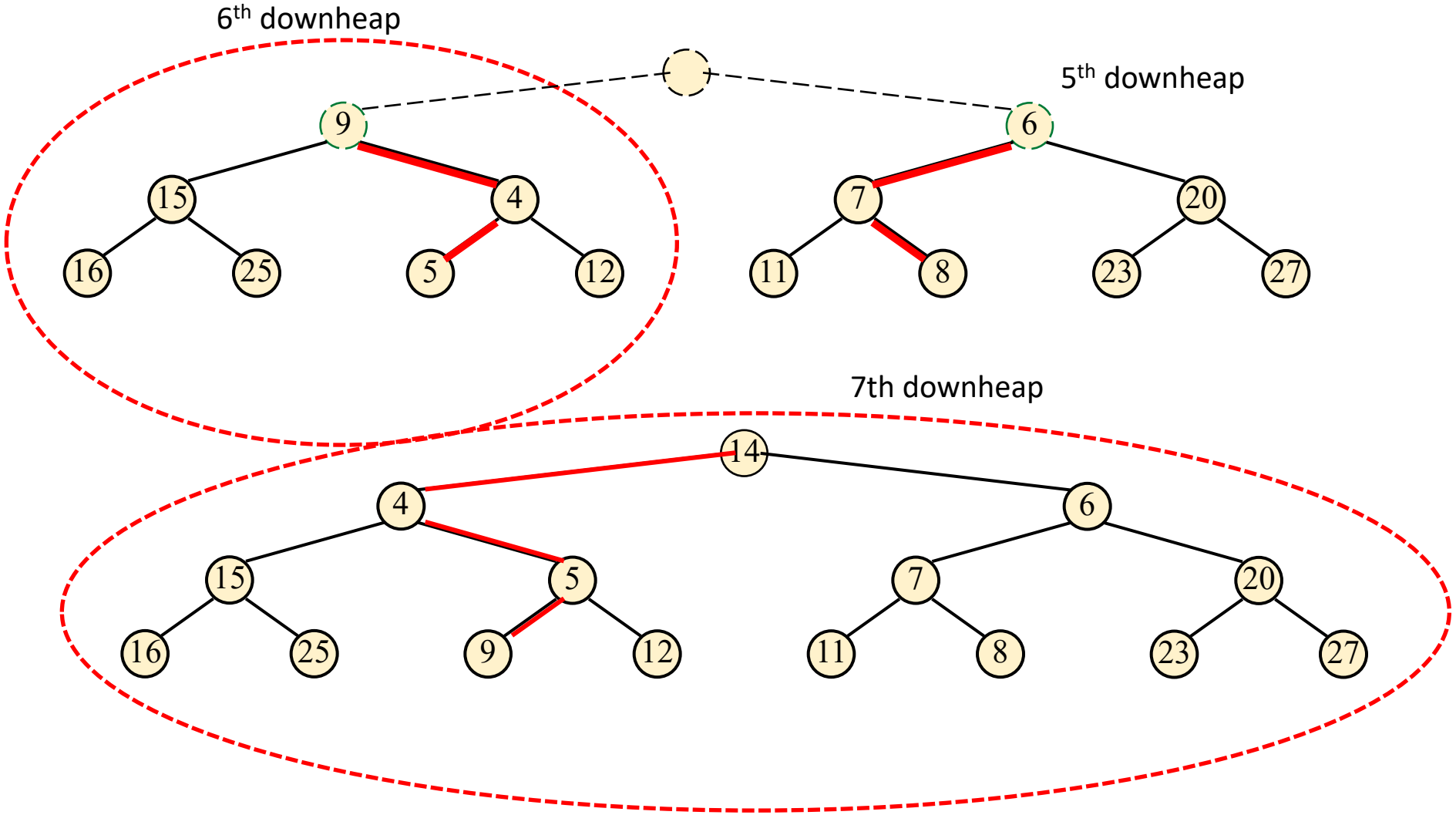
Construct a min-Heap using {14,9,8,25,5,11,27,16,15,4,12,6,7,23,20}





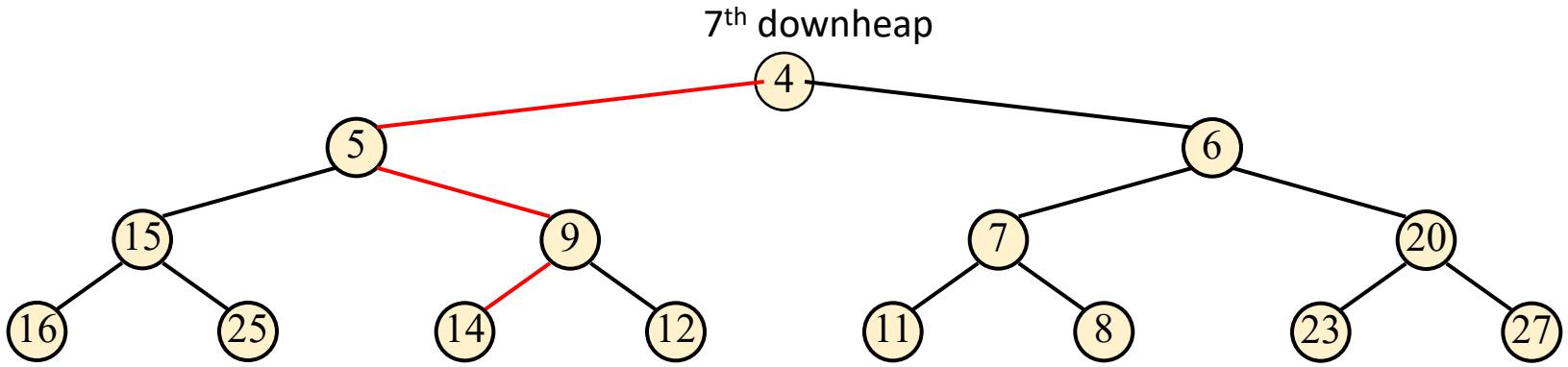
Exercise

Construct a min-Heap using {14,9,8,25,5,11,27,16,15,4,12,6,7,23,20}



Exercise

Construct a min-Heap using {14,9,8,25,5,11,27,16,15,4,12,6,7,23,20}



Final min heap:
{4,5,6,15,9,7,20,16,25,14,12,11,8,23,27}

Priority Queue and Heap: roadmap

- Priority queue ADT
- Implementing priority queues using lists
  - Unsorted list
  - Sorted list
- Heap
  - Insert and delete methods
  - Heap construction
- Heap sort

Sorting with a Priority Queue

- A Priority Queue P can be used for sorting a sequence S by:
  - inserting the elements of S one by one into P with a series of insert($e, \emptyset$) operations
  - removing the elements from P with a series of removeMin() operations and putting them back into S
- The running time of this sorting method depends on the priority queue implementation

Algorithm PQSort(S,C)

Input: list S, comparator C for the elements of S

Output: list S sorted in increasing order according to C

$P \leftarrow$ priority queue with comparator C

While !S.isEmpty() **do**

  $e \leftarrow S$.remove(S.first())

  P.insert($e, \emptyset$)

While !P.isEmpty() **do**

  $e \leftarrow P$.removeMin().getKey()

  S.addLast(e)

Unsorted list based implementation

Algorithm PQSort(S,C)

Input: list S, comparator C for the elements of S

Output: list S sorted in increasing order according to C

$P \leftarrow$ priority queue with comparator C

While !S.isEmpty() **do** ▷ $O(n)$

  $e \leftarrow S$.remove(S.first()) ▷ $O(n)$

  P.insert($e, \emptyset$) ▷ $O(n)$

While !P.isEmpty() **do** ▷ $O(n)$

  $e \leftarrow P$.removeMin().getKey() ▷ $O(n + (n-1) + \cdots + 1)$

  S.addLast(e) ▷ $O(n)$

- Sorting a sequence S with n elements
  - inserting the elements into the priority queue with n insert operations takes

$$O(n)$$

  - Selecting the elements in sorted order from the priority with n removeMin() operations takes time

$$n + (n-1) + (n-2) + \cdots + 1 \in O(n^2)$$

- Running time is $O(n^2)$

Sorted list based implementation

Algorithm PQSort(S,C)

Input: list S, comparator C for the elements of S

Output: list S sorted in increasing order according to C

$P \leftarrow$ priority queue with comparator C

While !S.isEmpty() **do** ▷ $O(n)$

  $e \leftarrow S$.remove(S.first()) ▷ $O(n)$

  P.insert($e, \emptyset$) ▷ $O(1 + 2 + \cdots + n)$

While !P.isEmpty() **do** ▷ $O(n)$

  $e \leftarrow P$.removeMin().getKey() ▷ $O(n)$

  S.addLast(e) ▷ $O(n)$

- Sorting a sequence S with n elements
  - inserting the elements in order with n insert operations takes

$$1 + 2 + \cdots + n \in O(n^2)$$

  - removing the n elements with removeMin() operations takes time

$$O(n)$$

- The running time is $O(n^2)$

Heap based implementation: Heap sort

- Heap sort: variation of PQSort that uses a heap to implement the priority queue P.

- Sorting a sequence S with n elements
  - building a heap using bottom-up construction takes $O(n)$
  - removing elements with n removeMin() operations takes time

$$\log n + \log(n-1) + \cdots + 1 \in O(n \log n)$$

- Heap sort runs in $O(n \log n)$ time

Significant improvement over $O(n^2)$!

Algorithm PQSort(S,C)

Input: array S, comparator C for the elements of S

Output: array S sorted in increasing order according to C

$P \leftarrow$ priority queue with comparator C

P.buildHeap(S) ▷ $O(n)$

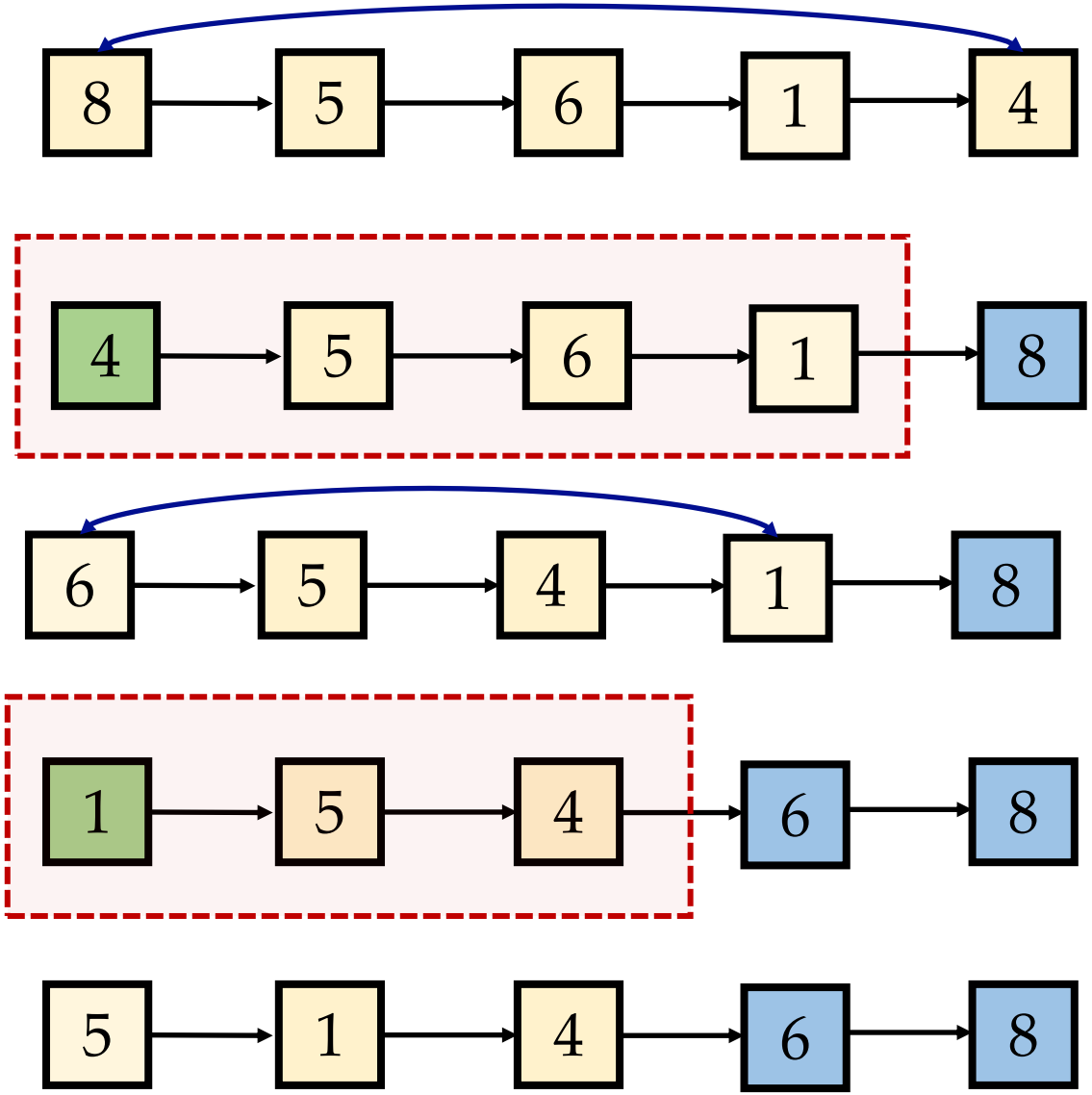
While !P.isEmpty() **do**

  $e \leftarrow P$.removeMin().getKey() ▷ $O(\log n + \log(n-1) + \cdots + 1)$

  S.addLast(e)

Heap sort in place

- Build array A into a **max**-heap
  - The first key $A[0]$ is the maximum and thus should be in the last position after being sorted
- Swap$(0, n-1)$
- Array $A[0, \ldots, n-2]$ is almost-heap. Run downheap to heapify it
- Repeat swap and heapify steps



Heap sort in place

Algorithm heapSort(A)

Input: array A

Output: sorted array A

buildHeap(A)

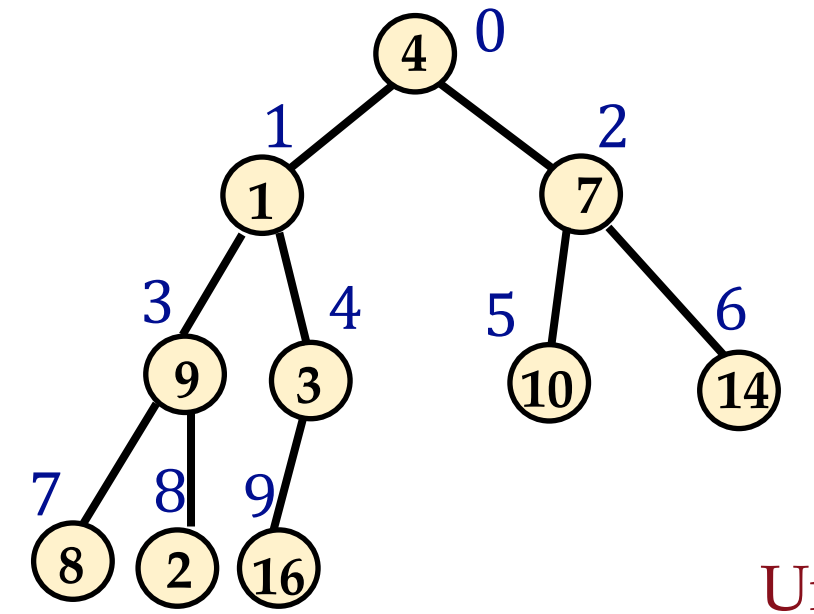
heapSize $\leftarrow A.size()$

For $i \leftarrow A.size() - 1$ down to 1 **do**

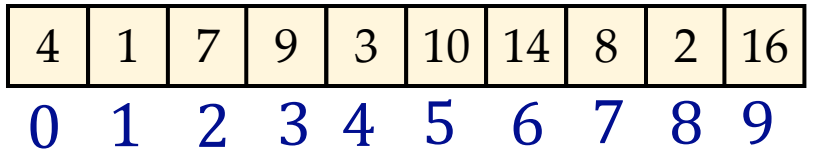
swap $(0, i)$

heapSize $\leftarrow$ heapSize $- 1$

downHeap $(0, \text{heapSize} - 1)$



Unsorted;
Not a heap



Heap sort in place

Algorithm heapSort(A)

Input: array A

Output: sorted array A

BuildHeap(A)

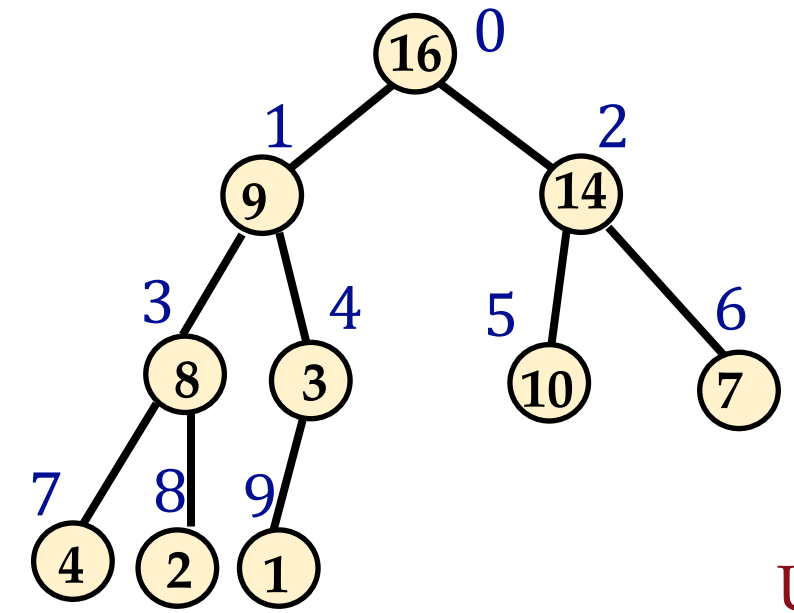
heapSize ← $A.size()$

For $i \leftarrow A.size() - 1$ down to 1 **do**

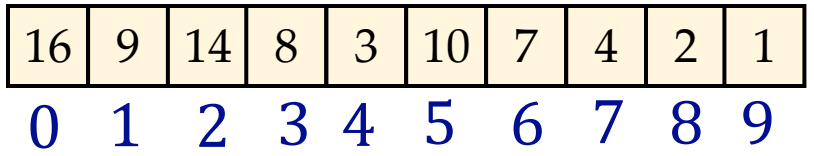
swap $(0, i)$

heapSize ← heapSize $- 1$

downHeap $(0, h\text{eapSize} - 1)$



Unsorted; Heap



Heap sort in place

Algorithm heapSort(A)

Input: array A

Output: sorted array A

BuildHeap(A)

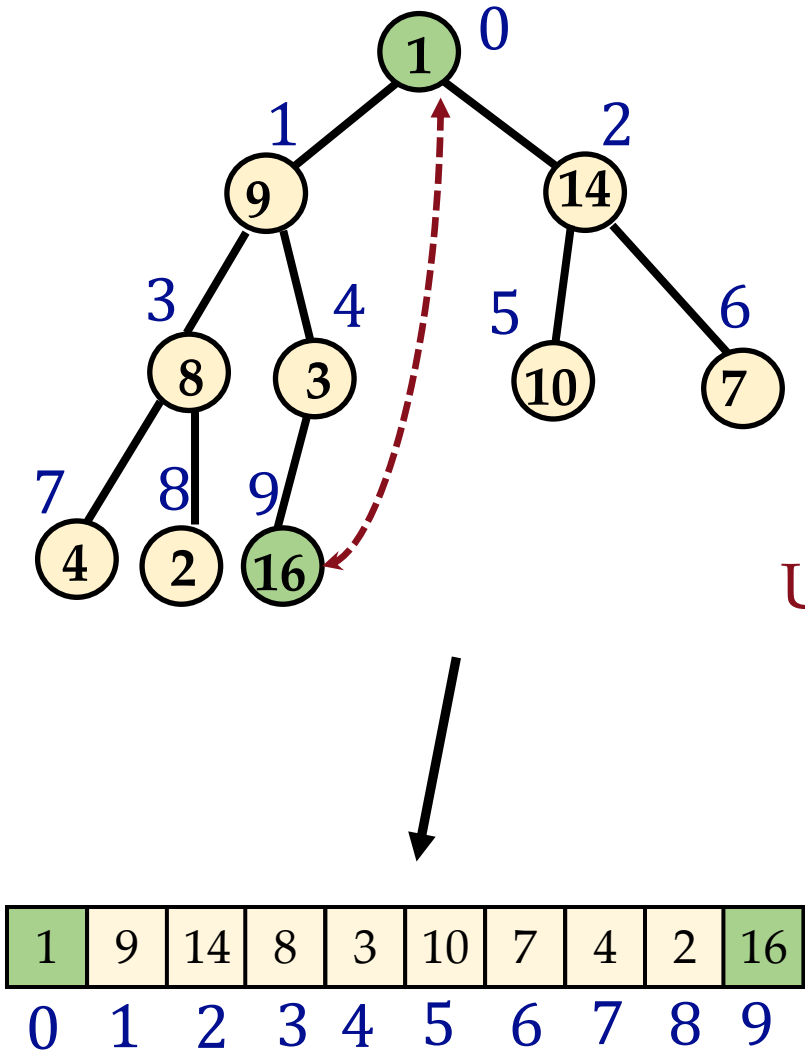
heapSize $\leftarrow A.size()$

For $i \leftarrow A.size() - 1$ down to 1 **do**

swap $(0, i)$

heapSize $\leftarrow$ heapSize $- 1$

downHeap $(0, heapSize - 1)$



Heap sort in place

Algorithm heapSort(A)

Input: array A

Output: sorted array A

BuildHeap(A)

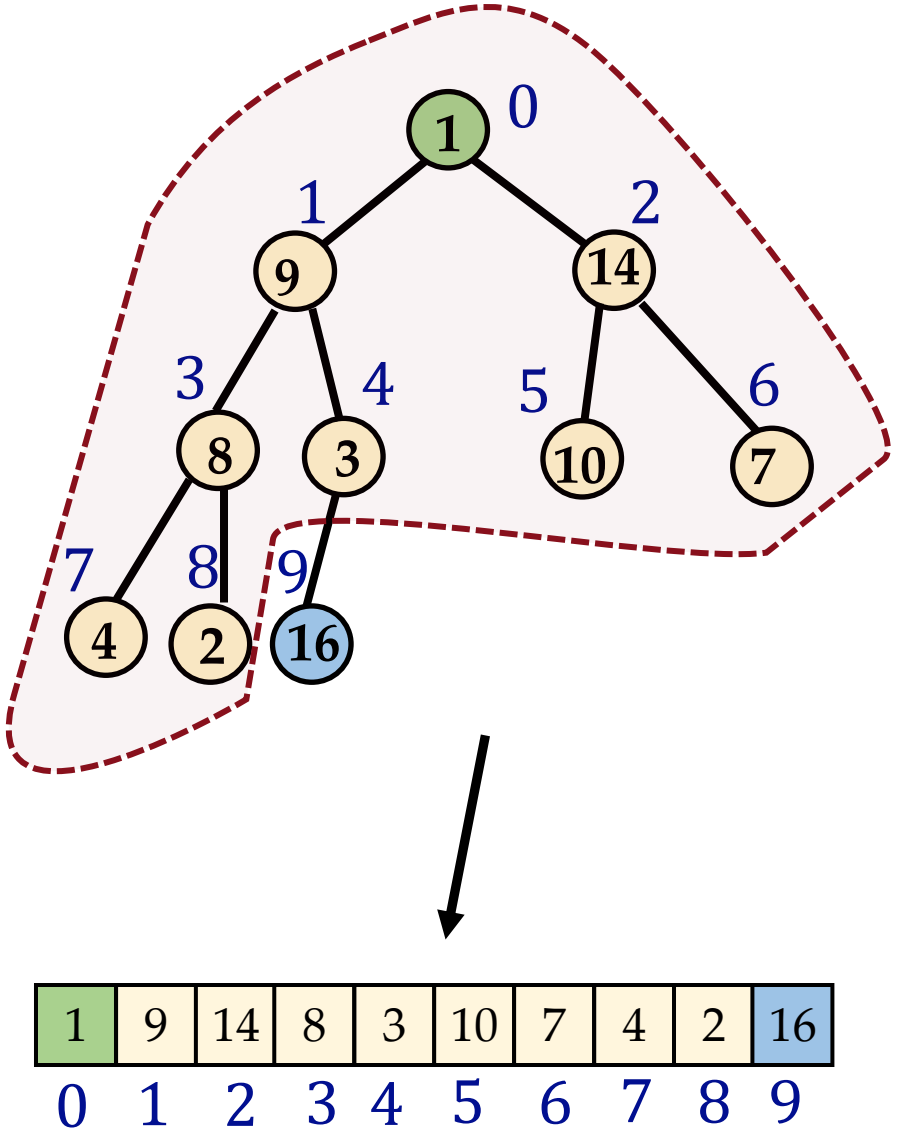
heapSize ← $A.size()$

For $i \leftarrow A.size(\ ) - 1$ down to 1 **do**

swap $(0, i)$

heapSize ← heapSize $- 1$

downHeap $(0, h\text{eapSize} - 1)$



Heap sort in place

Algorithm heapSort(A)

Input: array A

Output: sorted array A

BuildHeap(A)

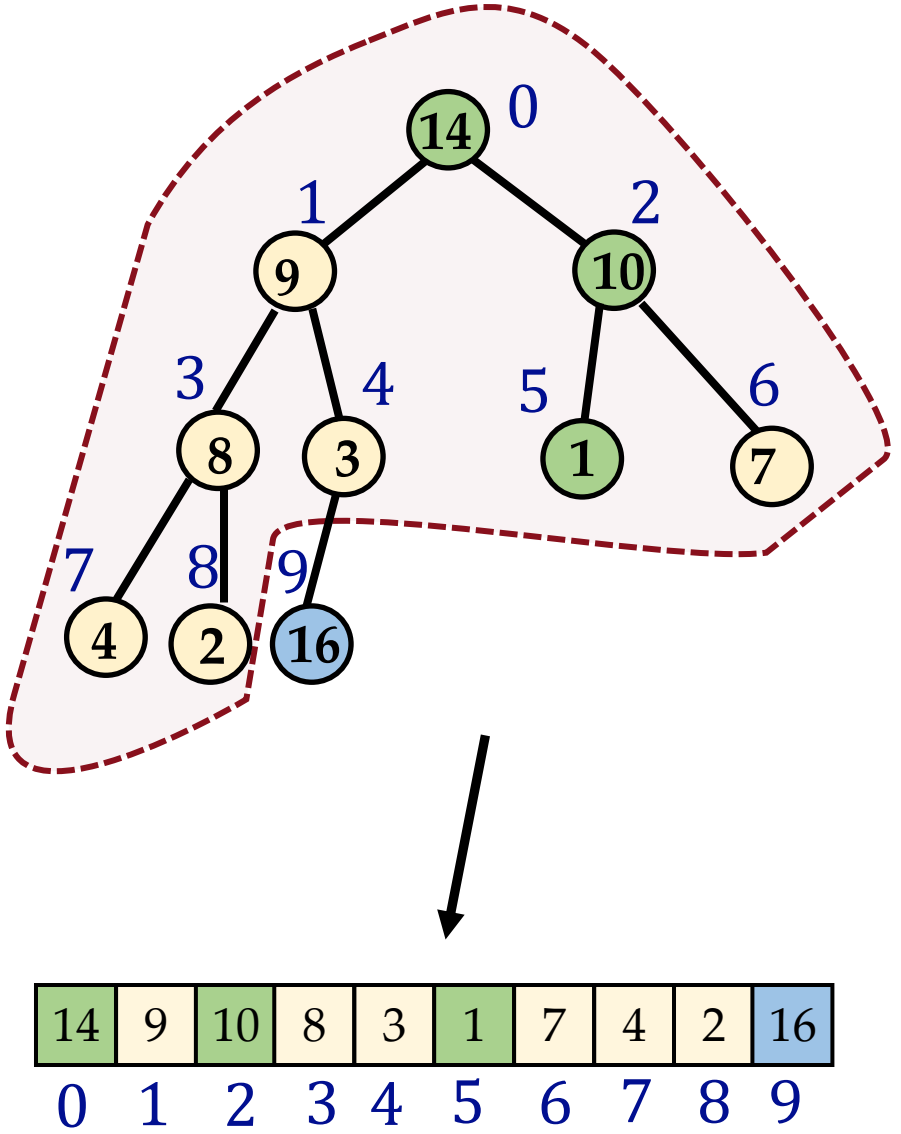
heapSize $\leftarrow A.size()$

For $i \leftarrow A.size(\ \ ) - 1$ down to 1 **do**

swap $(0, i)$

heapSize $\leftarrow$ heapSize $- 1$

downHeap $(0, heapSize - 1)$



Heap sort in place

Algorithm heapSort(A)

Input: array A

Output: sorted array A

BuildHeap(A)

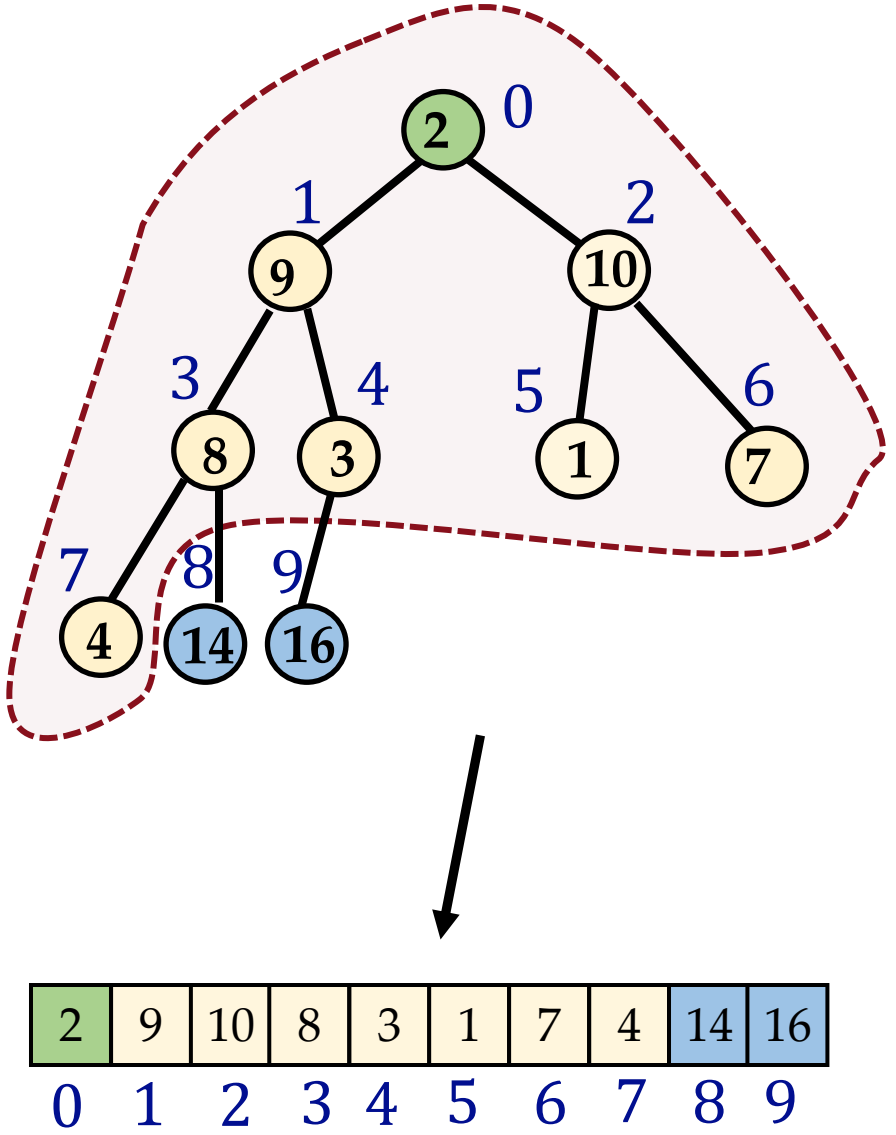
heapSize ← $A.size()$

For $i \leftarrow A.size(\ ) - 1$ down to 1 **do**

swap $(0, i)$

heapSize ← heapSize $- 1$

downHeap $(0, heapSize - 1)$



Heap sort in place

Algorithm heapSort(A)

Input: array A

Output: sorted array A

BuildHeap(A)

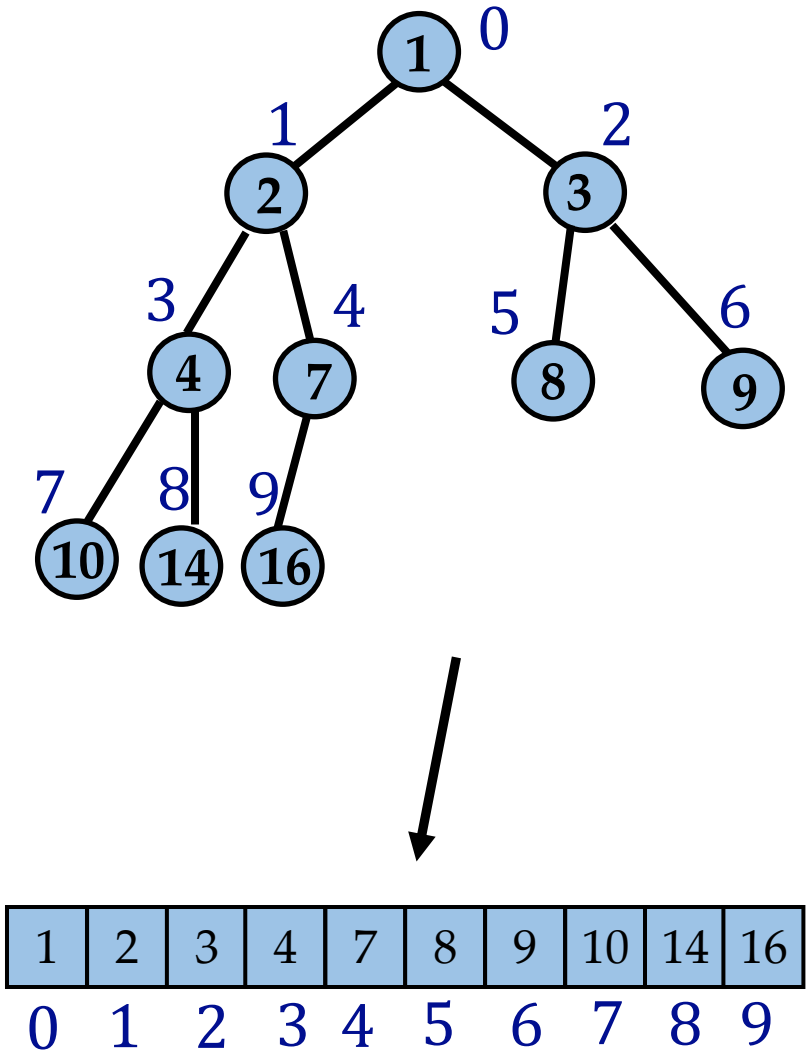
heapSize $\leftarrow A.size()$

For $i \leftarrow A.size(\ ) - 1$ down to 1 **do**

swap $(0, i)$

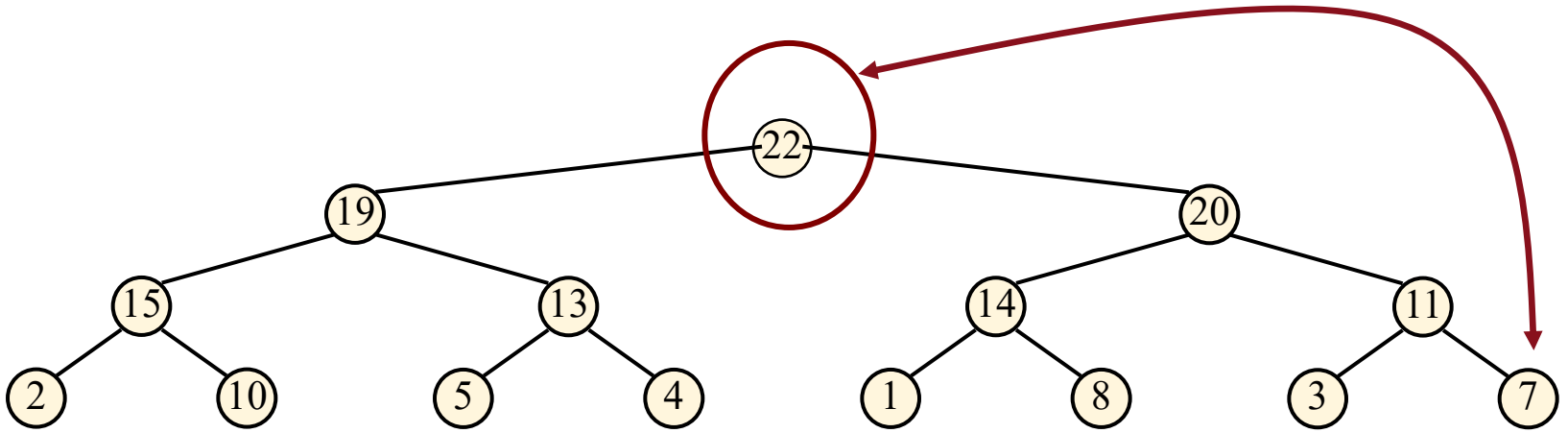
heapSize $\leftarrow$ heapSize $- 1$

downHeap $(0, h\text{eapSize} - 1)$



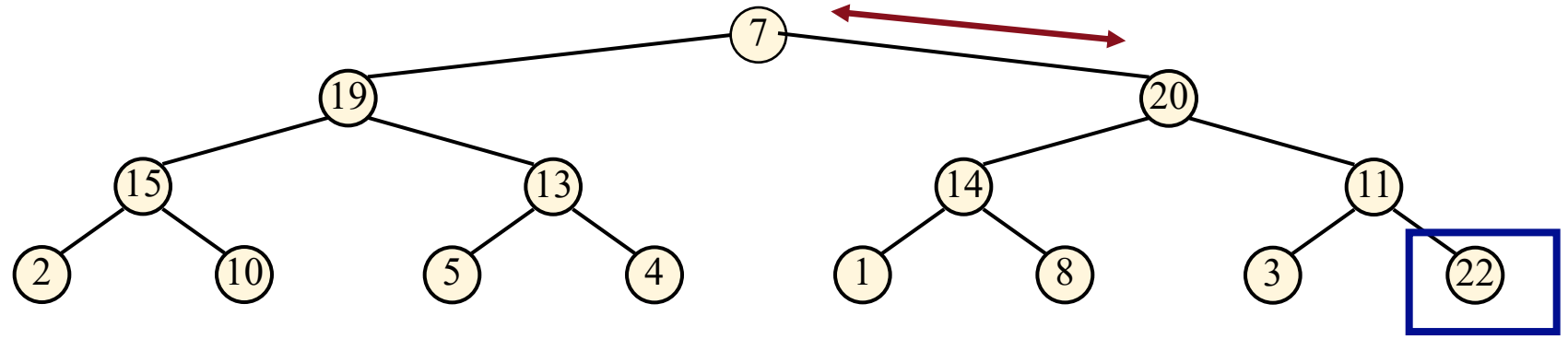
Exercise

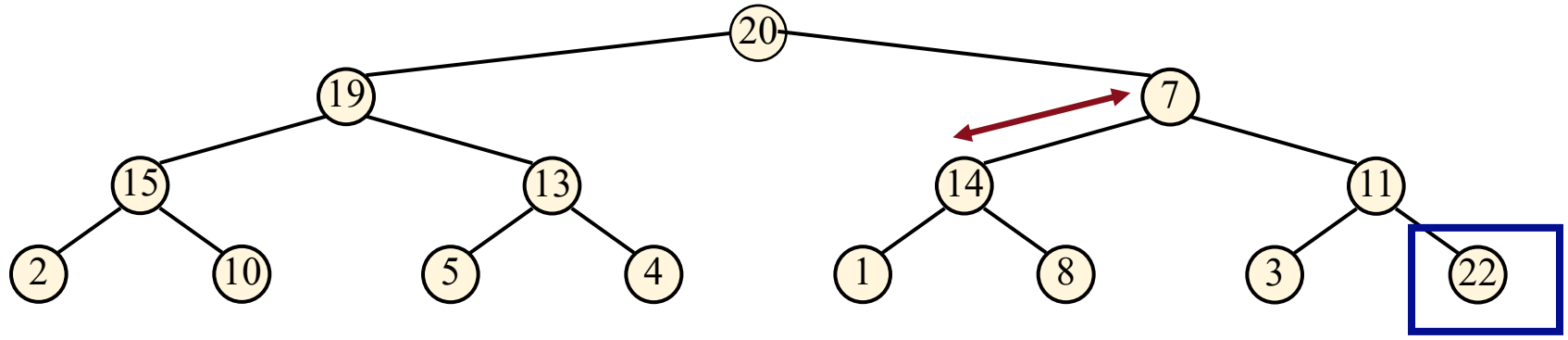
Sort the following heap



Exercise

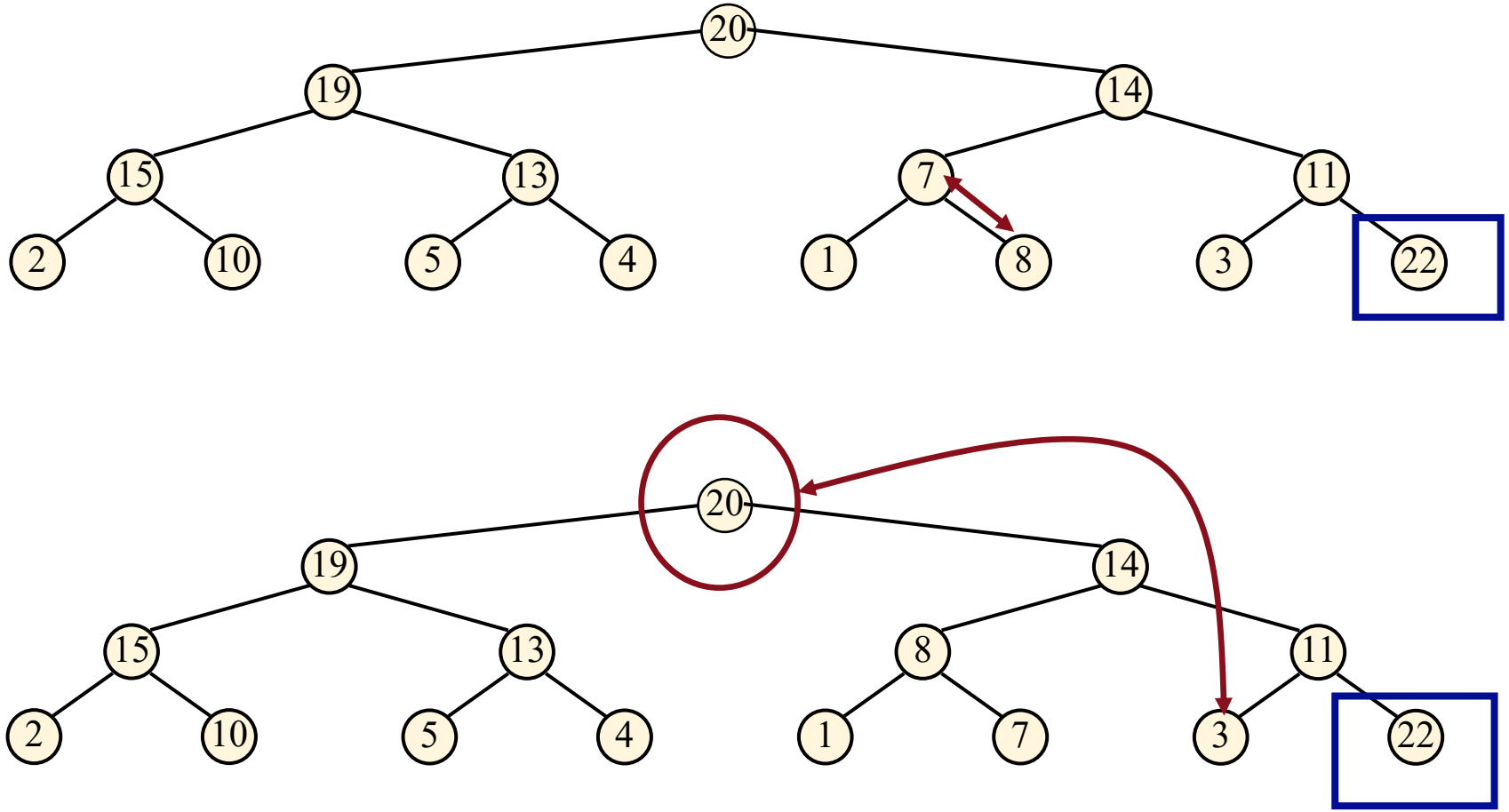
Sort the following heap



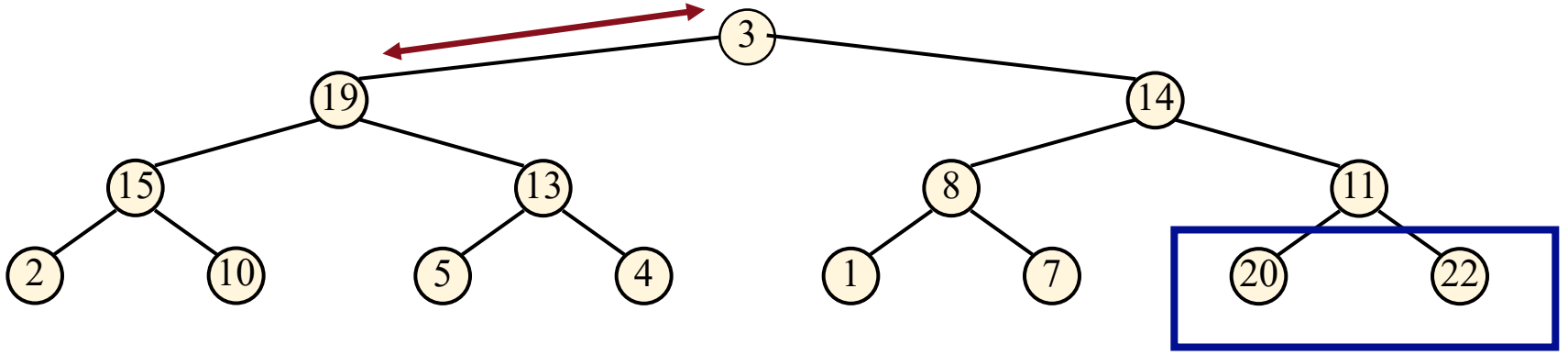


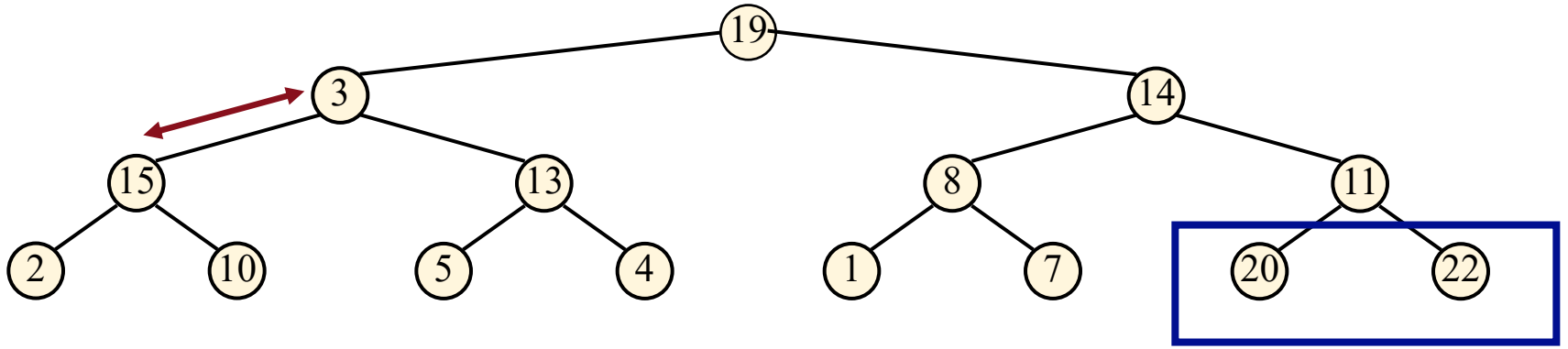
Exercise

Sort the following heap

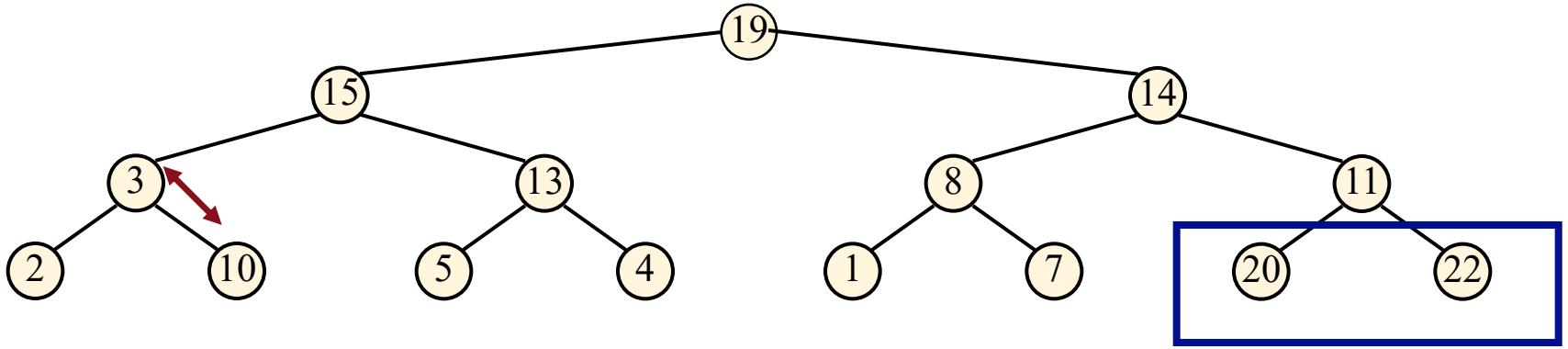


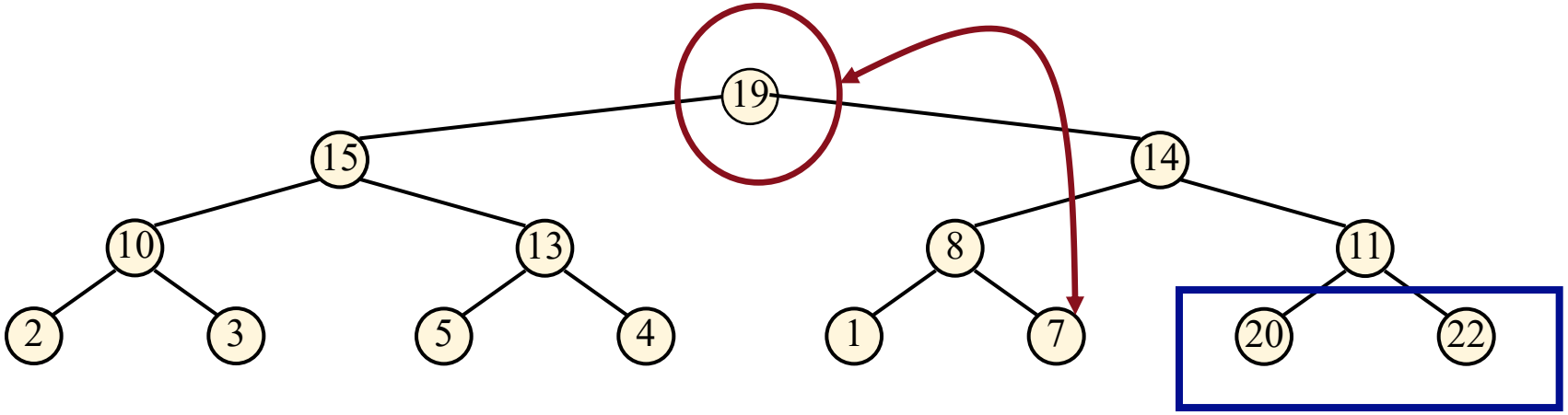
Exercise



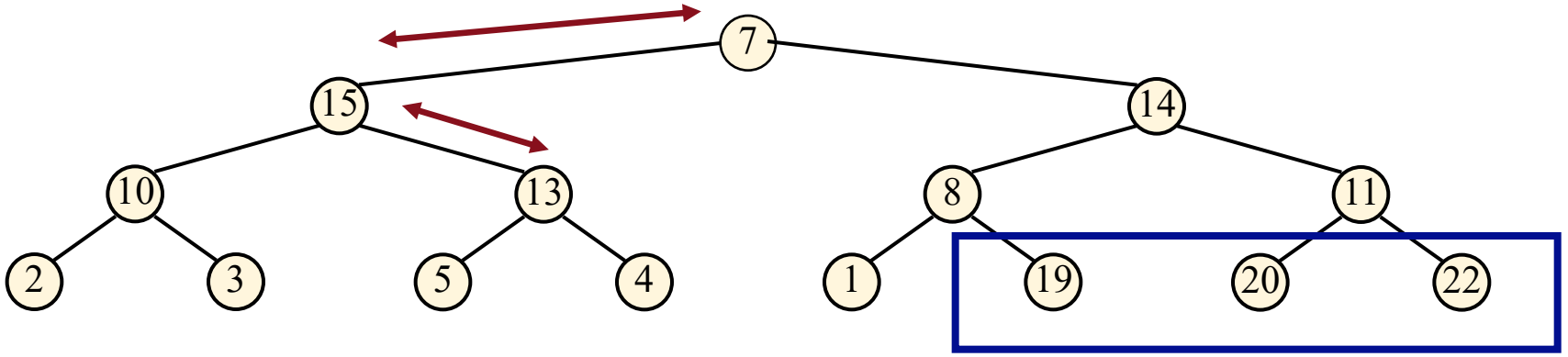


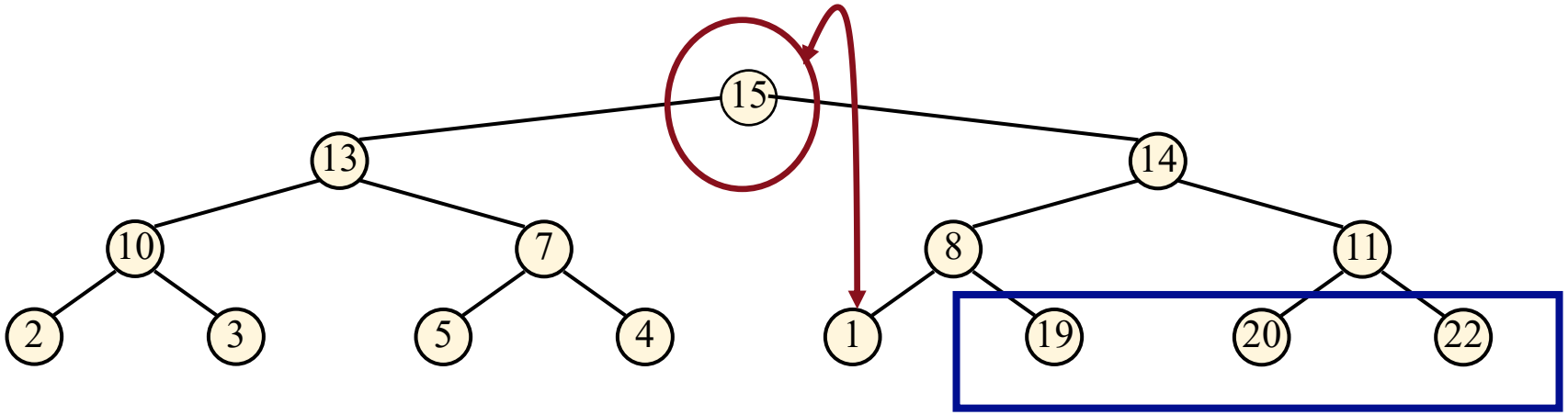
Exercise



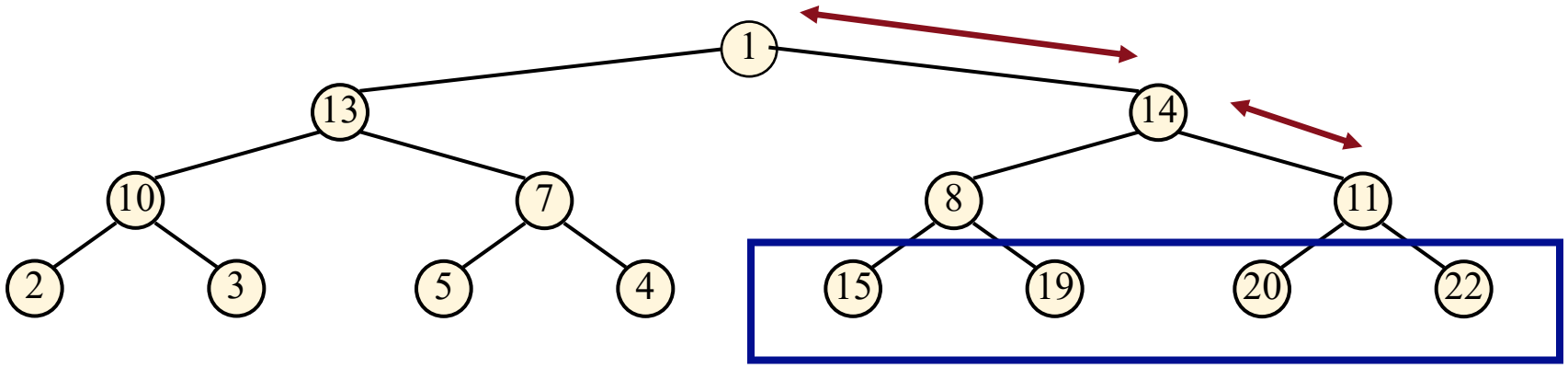


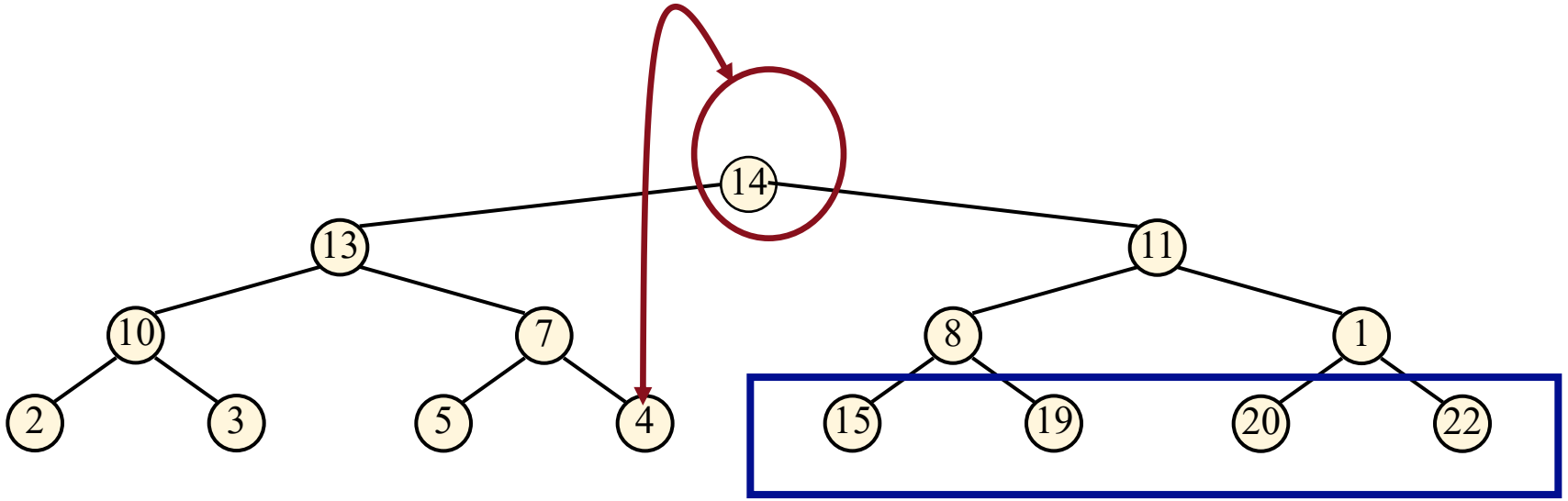
Exercise



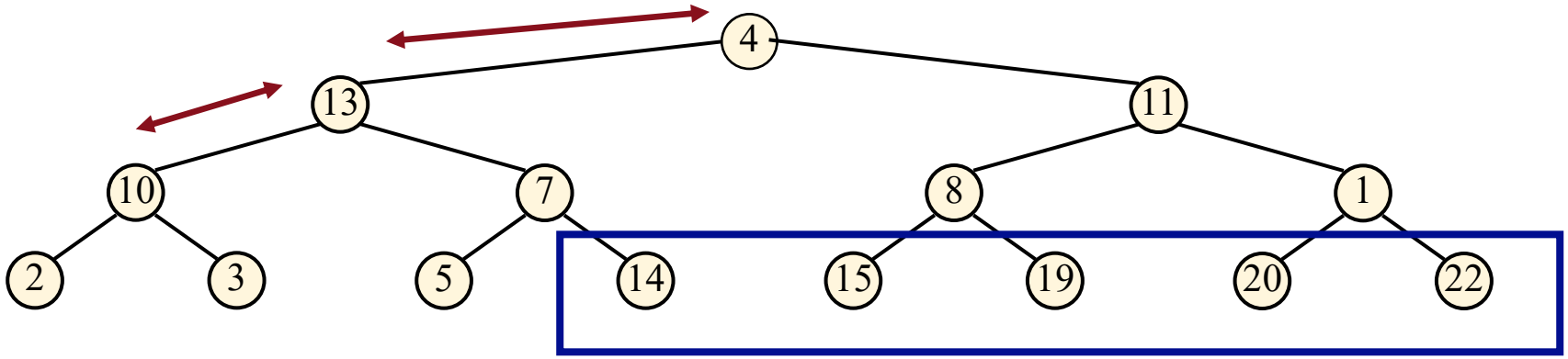


Exercise

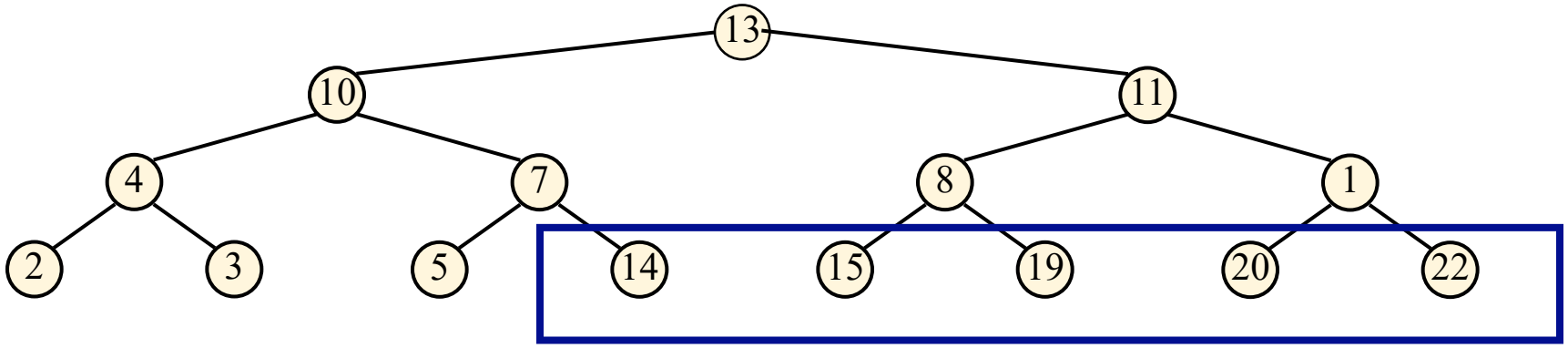




Exercise



Exercise



Heap part: unsorted

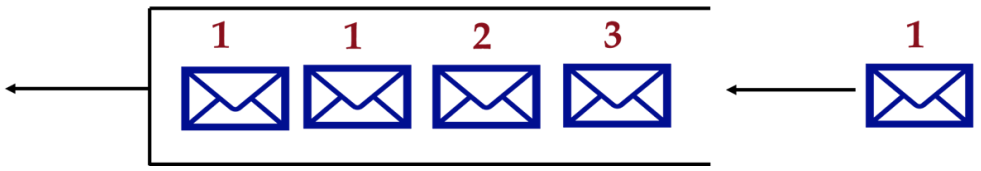
Sequence part: sorted

13	10	11	4	7	8	1	2	3	5	14	15	19	20	22

Try to continue this example

Summary

Priority queue; Sorting algorithms



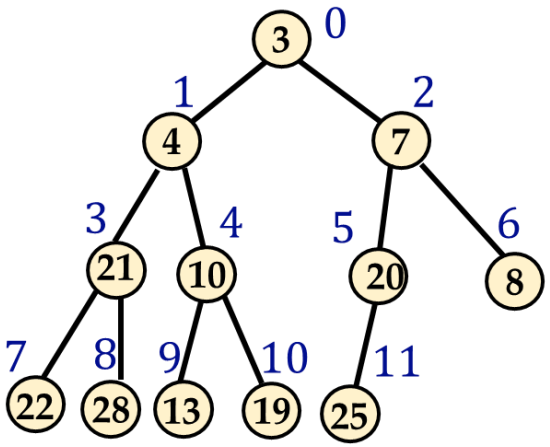
Methods	Unsorted list	Sorted list	Heap
insert()	O(1)	O(n)	O(log n)
min()	Θ(n)	O(1)	O(1)
removeMin()	Θ(n)	O(1)	O(log n)

```
Algorithm PQSort(S,C)
  Input: list S, comparator C
  for the elements of S
  Output: list S sorted in increasing
  order according to C
  P ← priority queue with
  comparator C
  While !S.isEmpty() do
    e ← S.remove(S.first())
    P.insert(e, ∅)
  While !P.isEmpty() do
    e ← P.removeMin().getKey()
    S.addLast(e)
```

PQSort implementation	Time efficiency
Unsorted list	$O(n^2)$
Sorted list	$O(n^2)$
Heap	$O(n \log n)$

In-place implementation!

Summary
Heap



3	4	7	21	10	20	8	22	28	13	19	25
0	1	2	3	4	5	6	7	8	9	10	11

Algorithm `downHeap(j)`

While hasLeft(j) **then**
  smallIndex ← left(j)
  If hasRight(j) and key(left(j))>key(right(j)) **then**
    smallIndex ← right(j)
  If key(smallIndex)>key(j) **then**
    return
  swap(smallIndex, j)
  j ← smallIndex

Algorithm `buildHeap(A)`

For i ← $\left\lfloor \frac{A.size()}{2} \right\rfloor - 1$ **down to 0 then**
  downHeap(i)

Algorithm `heapSort(A)`

Input: `array` A
Output: `sorted array` A
BuildHeap(A)
heapSize ← $A.size()$
For $i \leftarrow A.size() - 1$ `down to` 1 **do**
  swap $(0, i)$
  heapSize ← heapSize $- 1$
  downHeap $(0, heapSize - 1)$

Complete binary tree with heap-order property

Removal, insertion and construction

Heap sort in place

Next Topic

Binary search tree